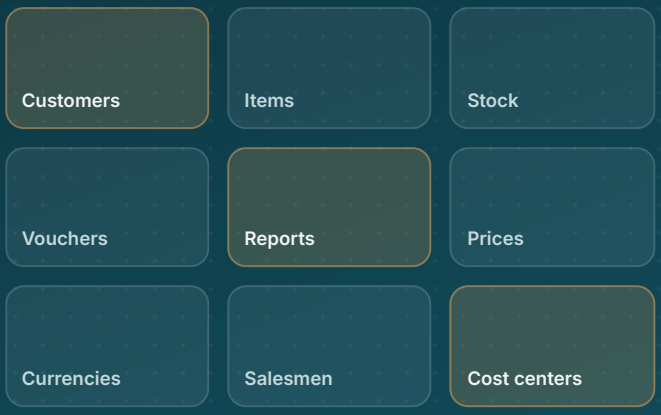


CAPABILITIES & SCOPE

Shamel Pro API

قدرات واجهة الشامل البرمجية ونطاقها

Everything ISCO's manual says the Shamel Pro Web API can do: the business data it exposes, what can be read and written, how far each capability goes, what we can build with it, and where its limits are.

Source
ISCO's Shamel Pro API manual (20 pages)

Date
3 November 2026

Prepared for
the Bawaba project

01 AT A GLANCE

A read-heavy API with insert-only writes

The Shamel Pro Web API gives programs broad access to the company's commercial data: customers, items, stock, prices, vouchers and reports. It can also create new records. The manual documents no way to change or delete them, and its only access control is a Shamel user login plus salesman scoping.

30
endpoints in the manual

25
read endpoints, including 3 reports

4
write endpoints (insert only), plus login

37 / 318
record types / fields documented

What you can do

- ✓ Read all customers (with getAll) with balances, cheque totals, contacts and price rows
- ✓ Read the full item catalog: up to 6 units and 6 sale-price fields, barcodes, groups, image paths
- ✓ Read stock per store, item serials and warranty dates
- ✓ Read vouchers by code or date range, the salesman's prepared delivery and order status
- ✓ Run account statements, last-price and cost-center reports
- ✓ Create customers, items, cost centers and vouchers of all 13 types listed for Post2
- ✓ Fetch customers and items updated after a date (stock takes the same header; effect unstated)

What you cannot do (today)

- ✗ Update, cancel or delete anything: only inserts are documented
- ✗ Page through big lists: no paging is documented; lists come back in one response
- ✗ Detect changes to vouchers or balances, or deletions, directly
- ✗ Restrict what an API user sees: no permissions are documented, and getAll lifts salesman scoping
- ✗ Rely on documented HTTPS, rate limits, token lifetime or versioning

★ **30 tool ideas checked against the manual:** 6 are possible with documented endpoints alone, 21 work with caveats or need an answer from ISCO, and the rest need something the API does not offer. See section 11.

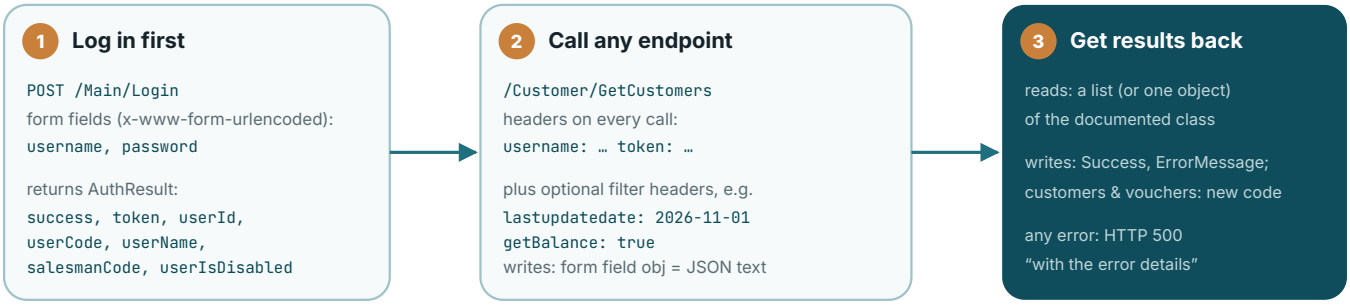
Contents

01 At a glance
02 How the API works
03 The data map
04 Capability matrix
05 Master data
06 Customers
07 Items, stock & catalog
08 Vouchers
09 Reports & cost centers
10 Writing data
11 What we can build
12 Limits & gaps
13 How Bawaba fills the gaps
A Endpoint reference
B Data dictionary
C Open questions

02 HOW THE API WORKS

Log in, then headers on every call

The Shamel Pro Web API is a small HTTP API published by ISCO next to the Shamel accounting program. You log in with a Shamel username and password, receive a token, and send it with your username on every later request. Read filters travel in HTTP headers (the account statements use a form field), and new records are posted as a JSON string inside a form field.



Filters are headers

Options such as `code`, `itemCode`, `getBalance` or `filters` (a JSON object) are sent as HTTP headers, not as URL parameters (p. 4–18).

Writes are form posts

The four write endpoints take a form field named `obj` holding a JSON string of the record (p. 8–20). The manual says "post parameter" for 7 endpoints in total.

Dates and sync

"Date format is always yyyy-MM-dd" (p. 4). Customers and items return records "updated after" `lastupdatedate`; the stock endpoint takes the same header, effect unstated.

Salesman scoping

Login returns the user's `salesmanCode`; customer lists are limited to that salesman unless `getAll=true` is sent (p. 4, 7).

Errors

"Any error occurred the service will return Http error 500 with the error details" (p. 4); write results also carry `Success` and `ErrorMessage`.

What the manual leaves out

No base URL, HTTPS, token lifetime, rate limits, paging, versioning or HTTP methods for 23 endpoints. These are in the ISCO question list.

Example: read one customer with balance

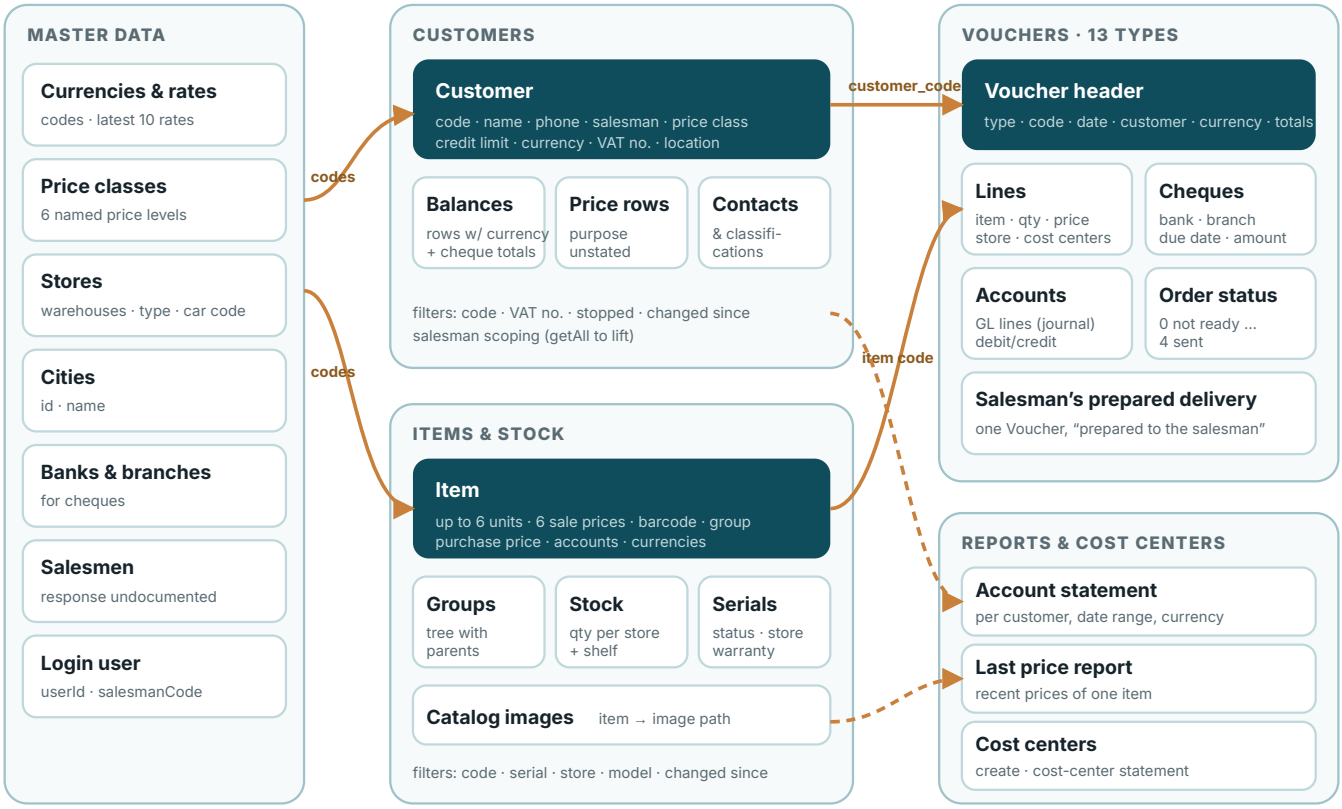
```
GET /Customer/GetCustomers   # method assumed
username: api-user
token: 9f8e…
code: C1001
getBalance: true
getContacts: true
```

```
# 200, a list of Customer (illustrative values)
[ { "code": "C1001", "name": "…",
    "salesman_code": "S07", "price_class": 2.0,
    "credit_limit": 5000.0, "currency": "NIS",
    "balance": 1250.5, "notPayedCheqsBalance": 0,
    "returnedCheqsBalance": 0, "contacts": [ … ], …
} ]
```

03 THE DATA MAP

What lives inside Shamel, and how it connects

The API exposes five areas of business data. Customers and items are the core records; vouchers (invoices, orders, receipts…) tie them together; master data supplies the codes they share; reports summarise the results.



37
record types described in the manual

318
fields across those records

13
voucher types, from orders to journal entries

6 + 6
unit fields and sale-price fields per item (meaning not documented)

04 CAPABILITY MATRIX

What you can do with each kind of data

✓ supported by a documented endpoint · ◖ partly (with limits or assumptions) · ✕ not documented. “Changed since” means the endpoint accepts lastupdatedate.

Data	List	Get one	Filter	Changed since	Create	Update	Delete	Filters
Currencies العملات	✓	✕	✕	✕	✕	✕	✕	
Exchange rates اسعار الصرف	◖	✕	✕	✕	✕	✕	✕	
Banks & branches البنوك وفروعها	✓	✕	✕	✕	✕	✕	✕	
Cities المدن	✓	✕	✕	✕	✕	✕	✕	
Stores المستودعات	✓	✕	✕	✕	✕	✕	✕	
Price classes فئات الاسعار	✓	✕	✕	✕	✕	✕	✕	
Salesmen المندوبين	◖	✕	✓	✕	✕	✕	✕	undocumented
Customers الزبائن	✓	✓	✓	✓	✓	✕	✕	code (one customer), vatreg, getStopped=true (stopped...
Customer balances ارصدة الزبائن	✓	✓	✓	✕	✕	✕	✕	customerCode (empty = all customers), vatreg
Customer special prices أسعار الزبائن الخاصة	◖	✕	✕	✕	✕	✕	✕	
Items الاصناف	✓	✓	✓	✓	✓	✕	✕	itemCode, serial, store, model, lastupdatedate (changed...
Item groups مجموعات الاصناف	✓	✕	✕	✕	✕	✕	✕	
Stock per store كميات الاصناف في المستودعات	✓	✓	✓	◖	✕	✕	✕	itemcode, lastupdatedate (listed, meaning not explained)
Item serials الأرقام التسلسلية للأصناف	✓	✓	✓	✕	✕	✕	✕	customer, serial
Item images كتالوج الاصناف	✓	✕	✕	✕	✕	✕	✕	
Vouchers (13 types) السندات	✓	✓	✓	✕	✓	✕	✕	fromDate + toDate (date range, presumably required),...
Orders / status طلبية مبيعات	◖	✓	✓	✕	✓	✕	✕	manual (presumably the order's manual_code)
Deliveries الارساليات	✕	◖	✕	✕	✓	✕	✕	
Account statement كشف حساب محاسبي	✓	✕	✓	✕	✕	✕	✕	customerCode, currencyCode, fromDate, toDate,...
Last price report تقرير اخر الأسعار	✓	✕	✓	✕	✕	✕	✕	ItemCode (mandatory), barcode (optional, inferred)
Cost centers مراكز التكلفة	✕	✕	✕	✕	✓	✕	✕	
Cost-center statement تقرير كشف حساب لمركز تكلفة	✓	✕	✓	✕	✕	✕	✕	fromCostcenter, toCostcenter, account, type (values...
GL accounts (chart of accounts) الحسابات	✕	✕	✕	✕	✕	✕	✕	
Other lookup codes قوائم الترميز الأخرى	✕	✕	✕	✕	✕	✕	✕	

! **The pattern:** almost everything can be read; four kinds of records can be created; nothing can be updated or deleted through the API. Corrections would have to be made in Shamel itself or with an offsetting voucher, for example a return or credit note (our reading).

05 MASTER DATA

The shared lists every other record refers to

Currencies and exchange rates, banks and branches, cities, stores, the six price classes and salesmen (response undocumented). Mostly short lookup lists (exchange rates change more often); tools use them to translate codes into names and to fill drop-downs.

Endpoints

Endpoint	What it returns	Inputs & filters	Sync
`/Main/Login`	Log a Shamel user in and get the session token that every other call must carry.	password	
`/Files/GetCurrencies`	List all currencies defined in Shamel.	—	
`/Files/GetCurrencyRates`	Get the latest 10 exchange rates.	—	
`/Files/GetBanks`	List all banks.	—	
`/Files/GetBanksBranchs`	List all bank branches.	—	
`/Files/GetCities`	List all cities.	—	
`/Files/GetStores`	List all stores (warehouses).	—	
`/Files/GetPriceClasses`	Get the names of the 6 price classes.	—	
`/Files/GetSalesmen`	Presumably lists salesmen; the manual prints only the heading.	—	

Records

Login result · 7 fields · p. 4

What Shamel sends back when a user logs in with username and password (form POST, p. 4). It carries the session token that every other request must send, plus who the user is and which salesman they are linked to.

```
success · token · userId · userCode · userName · salesmanCode · userIsDisabled
```

Use it for: Lets a tool sign in once and act as a specific Shamel user; the user's salesman link decides which customers it sees by default.

Currency · 5 fields · p. 4

A currency defined in Shamel ("Get all currencies", pp. 4–5). It is the lookup list that amounts, prices and balances point to through the currency code.

```
id · code · currency_name · units · division
```

Use it for: Lets tools show every amount with the right currency and match vouchers, prices and balances to it.

Exchange rate · 4 fields · p. 5

An exchange-rate entry. The endpoint returns only the "latest 10 currency rate" (p. 5) with no date or currency filter, so it gives current rates, not history. Whether that is 10 in total or 10 per currency is not stated (ISCO-28 c).

```
currency_code · rate · date · is_base
```

Use it for: Lets tools convert foreign-currency balances and prices at the latest rate; historical or as-of-date rates are not available.

Bank · 2 fields · p. 5

A bank defined in Shamel ("Get All Banks", p. 5; Arabic capability "سحب البنوك وفروعها", pull banks and their branches, p. 2). It identifies the bank of cheques.

```
code · name
```

Use it for: Lets tools pick a valid bank when recording cheques on receipts and payments.

05 MASTER DATA · CONTINUED

The shared lists every other record refers to

Bank branch · 3 fields · p. 5

A branch of a bank ("Get all branchs" [sic], p. 5). The path is misspelled /Files/GetBanksBranchs; it returns all branches of all banks and cannot be filtered by bank.

```
code · name · bank_code
```

Use it for: Completes cheque details (bank plus branch) for receipts and payments.

City · 2 fields · p. 6

A city in Shamel's city list ("Get all cities", p. 6). Customers and vouchers point to it by number.

```
id · name
```

Use it for: Lets tools group and filter customers by city and fill the city on new customers.

Store (warehouse) · 5 fields · p. 6

A store (warehouse) where stock is held ("Get all stores", p. 6; Arabic capability "سحب المستودعات", pull warehouses, p. 2). The manual prints this class as `class City`, a copy-paste error; "Store" is our name for it.

```
id · name · type · car_code · orgnization_code
```

Use it for: Lets tools show stock per warehouse or van and choose the right store on orders and invoices.

Price class · 2 fields · p. 6

One of Shamel's six named price levels ("Get the 6 price classes name", p. 6; Arabic capability "سحب فئات الاسعار", p. 2). Each customer is assigned one.

```
id · name
```

Use it for: Lets tools show which price list applies to a customer and label item prices by class.

Salesman · 0 fields · p. 6

A sales representative (مندوب). UNDOCUMENTED: the manual prints only the heading /Files/GetSalesmen (p. 6) with no description, no inputs and no fields, and the endpoint is missing from the Arabic capability list (p. 2). Its response must come from ISCO or a live capture (ISCO-26, priority).

Use it for: Would let tools list salesmen and report per salesman. Salesman codes already appear in AuthResult.salesmanCode, Customer.salesman_code, Customer(write).Salesman and VoucherHeader.salesman ("رقم المندوب", salesman number, p. 13), but the list itself cannot be relied on until ISCO answers.

06 CUSTOMERS

Who you sell to, what they owe, and what they pay

Customer profiles with salesman, price class, credit limit, currency and location; a current balance with unpaid and returned cheque totals, plus a balance list whose rows each carry a currency; classifications, contacts and customer price rows (purpose not described).

Endpoints

Endpoint	What it returns	Inputs & filters	Sync
`/Customer/GetCustomers`	List customers (by default only the logged-in salesman's), optionally only those changed since a date, with optional balances, classifications and contacts.	lastupdatedate, code, vatreg, getBalance, getStopped, getClassifications, getContacts, getAll	✓
`/Customer/GetCustomerBalances`	Get the balances of one customer (by code or VAT number), or of all customers.	customerCode, vatreg	
`/Customer/GetCustomerPricesAsRows`	Presumably returns customer- or price-class-specific item prices and discounts as rows; the manual states no purpose.	—	

Records

Customer (read) · 33 fields · p. 7

A customer account as returned by /Customer/GetCustomers (pp. 6–7): identity, phone and address, salesman, price class and credit terms, tax status, GPS, and optionally balances, classifications and contacts. By default only the logged-in user's salesman's customers come back; getAll=true returns all (p. 7). Filters: lastupdatedate (changed since), code, vatreg, getStopped.

`code · name · phone · street · street2 · referee1 · referee2 · salesman_code · price_class · credit_limit · discount · city · currency · level · town · city_name · citizen_code · attr · +15 more`

Use it for: Gives a full customer list for CRM, route planning (GPS), credit control (limit vs balance) and incremental sync of changed customers via lastupdatedate.

Customer classification · 2 fields · p. 7

A classification tag attached to a customer (e.g. a segment or group). Returned inside Customer.classifications only when getClassifications="true" (p. 7). The class name is misspelled in the manual ("Classificaiton").

`pos · name`

Use it for: Lets tools segment customers for reporting and targeting; no endpoint lists all possible classifications.

Customer contact · 5 fields · p. 8

A contact entry for a customer (a person, phone, email or similar). Returned inside Customer.contacts only when getContacts="true" (p. 7).

`pos · name · detail · type · typeName`

Use it for: Gives extra phone numbers, emails and contact persons for calling or messaging customers.

Customer balance · 6 fields · p. 8

A customer balance row ("Get list of customer balances", p. 8; Arabic capability "سحب ارصدة الزبائن", p. 2). Apparently one row per customer per currency. Empty customerCode returns all customers' balances; vatreg finds a customer by VAT number.

`code · pos · currency · date · balance · rate`

Use it for: Lets tools show who owes what, per currency, for collections and credit control. Whether it is limited to the salesman's customers is unknown (ISCO-25).

Who you sell to, what they owe, and what they pay

Customer (new-customer form) · 13 fields · p. 9

The data sent to create a customer via /Customer/Post ("Insert new customer", p. 8), as a JSON string in form field `obj`. Same class name as the read model but different (PascalCase) fields; there is no customer code because Shamel assigns it and returns it in PostResponse.value (p. 9). All meanings come from Arabic comments.

```
Name · VatReg · Address · MobileNumber · PhoneNumber ·
Salesman · Street · City · Town · Referee ·
RefereeMobile · CustomerLevel · PriceClass
```

Use it for: Lets tools (a sales app or web form) create new customers directly in Shamel. Insert-only: no update or delete; currency, credit limit and discount cannot be set (ISCO-48 a); a retried insert may create a duplicate (ISCO-43).

Insert result · 3 fields · p. 9

The JSON result of a customer or item insert ("It return the following class as Json", pp. 9 and 20): success flag, error text and the new record's code.

```
Success · ErrorMessage · value
```

Use it for: Confirms a new customer was created and gives its Shamel code for later orders, invoices and statements.

Customer price row · 11 fields · p. 16

A special price/discount row from Customer/GetCustomerPricesAsRows (pp. 16–17, path printed without leading slash). The manual gives no class name ("CustomerPriceRow" is our name), no purpose and no inputs, only "Return the following object". The fields suggest item prices and discounts per customer or price class. Not in the Arabic capability list (p. 2).

```
code · price_class_name · customer_code · stock_code ·
discount · discount_type · price · currency_code ·
main_unit · orgnization_code · tax
```

Use it for: Would let tools quote each customer their agreed price and discount; with no documented filters each call may download the whole price table, and list-vs-single result is unclear (ISCO-33).

07 ITEMS, STOCK & CATALOG

What you sell, where it is, and at what price

The item catalog with up to six units and six sale prices per item, barcodes, groups and accounting links; stock quantities per store; serial numbers with warranty dates; and image paths for the catalog.

Endpoints

Endpoint	What it returns	Inputs & filters	Sync
`/Item/GetItems`	List items (all, or only those changed since a date), optionally filtered by item code, serial, store or model.	lastupdatedate, serial, itemCode, store, model	✓
`/Item/GetItemsStores`	Get stock quantities per item per store, for all items or one item.	lastupdatedate, itemcode	✓
`/Item/getItemDetails`	Get one item together with its quantities in each store.	itemCode	
`/Item/GetItemsGroups`	List item groups (a tree via parent codes).	—	
`/Item/GetItemsCatalog`	List the image path of every item (the item catalogue).	—	
`/Item/GetItemsSerials`	List serial-numbered item units, optionally for one customer or one serial number.	customer, serial	

Records

Item / product (read) · 51 fields · p. 9

The full item (product) card as Shamel holds it: names, up to 6 units with conversion factors, 6 sale prices with 6 currency fields, 6 per-unit prices, barcode and other codes, group, status flags, purchase price and two GL accounts (pp. 9-10, 51 fields). It is returned as a list by /Item/GetItems and embedded as ItemDetails.itm by /Item/getItemDetails; every stock, catalog, serial and sales record joins to it by code.

```
code · name · nameEn · unit1 · unit2 · unit3 · unit4 ·
unit5 · unit6 · unit2_relation · unit3_relation ·
unit4_relation · unit5_relation · unit6_relation ·
sale_parice1 · sale_parice2 · sale_parice3 ·
sale_parice4 · +33 more
```

Use it for: Build a product list or customer-class price list, check units, status and barcodes, and pull only changed items with the lastupdatedate header (GetItems also filters by serial, itemCode, store and model headers, p. 9).

Stock quantity of an item in one store · 5 fields · p. 11

One row per item per store (warehouse or van) with the quantity, shelf and unit (p. 11). Returned in bulk by /Item/GetItemsStores ('Get all items quantities', p. 10; filters: lastupdatedate and itemcode headers, no store filter) and per item as ItemDetails.itmStores.

```
item_code · store · qnty · shelf · main_unit
```

Use it for: See stock per warehouse or van, spot shortages or overstock, and tell staff which shelf an item is on.

One item with its stock per store · 2 fields · p. 11

A wrapper holding one item's full card plus its quantity in every store, looked up with the itemCode header (p. 11); it serves the p. 2 capability 'pull quantities of a specific item' (سحب كمات [sic] صنف معين). The manual says 'It return list of this structure' but the shape is a single object, so callers must accept either (Q32).

```
itm · itmStores
```

Use it for: Answer 'what is this item, what does it cost, and how much do we have where?' in a single call.

Item group (category) · 4 fields · p. 11

A node in the item category tree with its parent code and level (p. 11). Items point to it through item_group_code and item_group_father_code. /Item/GetItemsGroups has no inputs and is not in the p. 2 capability list.

```
code · name · parentCode · level
```

Use it for: Organise products into categories for menus, filters and sales-by-category reports.

What you sell, where it is, and at what price

Item image reference · 2 fields · p. 11

Pairs an item code with the location of its picture ('Get all items image', p. 11). Only a path string comes back; its format, and how to fetch the image itself, are undocumented (ISCO-34). /Item/GetItemsCatalog has no item filter.

`item_code · image_path`

Use it for: Show product photos in a catalogue or ordering app, once ISCO confirms what image_path holds.

Serial-numbered unit of an item · 7 fields · p. 12

One serial-numbered unit of an item with its status, store and warranty (guarantee) start and end dates (p. 12). Returned by /Item/GetItemsSerials ('Get all items with serials only', p. 11), filterable by customer code and serial headers; not in the p. 2 capability list.

`name · itemCode · serial · status · store · start_grantee_date · end_grantee_date`

Use it for: Track individual devices by serial number, check warranty dates for after-sales service, and list serials linked to a customer.

Last-price report row (item pricing history) · 8 fields · p. 12

One row of the 'last prices' report (تقرير اخر الأسعار, p. 3) for one item: date, document code, price, currency, quantity, bonus, unit and customer (p. 12). Inputs are headers ItemCode ('mandatory') and barcode; the manual's 'ItemCode is optional' next to barcode contradicts itself (Q21, ISCO-37a).

`date · code · price · currency · quantity · bonus · unit · customer`

Use it for: Before quoting, see what price an item recently went for, in which unit and currency, and to whom.

New item (write) · 15 fields · p. 20

The JSON object sent in form field obj to /Item/Post to 'Insert new Item' (pp. 19-20); the manual prints its class name as plain 'Item'. Every field has an Arabic comment; it is a smaller, differently named set than the read model (one unit, one price, one currency), and the caller supplies the item code.

`code · unit1 · name · type · price1 · currency · group · barcode · original_code · factory_code · trademark · producing_company · style · model · note`

Use it for: Create new products in Shamel from another system (online shop, supplier price list) without retyping; insert only, with no update or delete endpoint (Q43, ISCO-23).

08 VOUCHERS

Every commercial document, from order to journal entry

Vouchers are Shamel's documents: orders, invoices, returns, deliveries, receipts, payments, notes and journal entries. Each can carry a header, item lines, cheques and account lines. They can be read by code or by date range.

Endpoints

Endpoint	What it returns	Inputs & filters	Sync
`/Voucher/GetDelivery`	Get the delivery that has been prepared for the logged-in salesman.	—	
`/Voucher/GetStatus`	Get the status of an order: not ready, ready, closed, sent partially or sent.	manual	
`/Voucher/GetVoucher`	Fetch one voucher (header, lines, cheques, accounts) by its code, optionally with the customer's balance at that time.	voucher_code, getCustomerBalance	
`/Voucher/GetVouchers`	List vouchers in a date range, optionally for one customer and/or currency.	filters, getCustomerBalance	
`/Voucher/GetExtraVouchers`	Presumably a lightweight list of vouchers in a date range with amount, direction and the customer's balance at each voucher date; the manual does not explain it.	filters, getCustomerBalance	

Records

Voucher (complete document) · 4 fields · p. 13

One accounting or stock document of any of the 13 voucher types (invoice, return, receipt, payment, order, journal, delivery…), made of a header plus item, cheque and account lines. Printed as class Voucher on p. 13 (Post2 input) and p. 17 (GetVoucher output); GetDelivery (p. 16) returns a variant without accounts, GetVouchers says only 'array of Voucher' without naming the class (ISCO-40a), and GetExtraVouchers uses a different short class of the same name (see ExtraVoucher).

`header · items · cheqs · accounts`

Use it for: The heart of any integration: one structure lets tools create and read sales invoices, returns, receipts, payments, orders and journal entries in Shamel.

Voucher header · 26 fields · p. 13

The top part of every voucher: its type, number, date and time, currency, customer, salesman, totals and status flags (pp. 13–14). The same header is used for all voucher types; some fields only apply to orders or invoices.

`type · code · date · time · currency · rate · customer_code · city · manual_code · is_posted · is_printed · discount · balance · update_date · note · note2 · vat · delivery_id · +8 more`

Use it for: Carries the who, when and how-much of each document, so tools can create or display invoices, receipts and orders with customer, salesman, currency and totals.

Voucher item lines · 23 fields · p. 14

One line per item in a voucher: item, unit, store, quantity, price, bonus, discount, tax and up to six cost centers (p. 14). Some fields are filled by Shamel only (voucher_code, name) and some are not used (measure, color, serials).

`voucher_code · pos · code · name · unit · store · measure · color · quantity · price · bonus · discount · vat · sold_qnty · delivery_pos · note · serials · costcenter1 · +5 more`

Use it for: Line-level detail (items, quantities, prices, discounts, tax, cost centers) needed for invoices, orders, returns and stock movements.

Voucher cheques · 8 fields · p. 14

One line per cheque attached to a voucher: cheque number, account, bank, branch, due date, currency and amount (p. 14).

`cheq_num · account_num · bank_code · branch_code · due_date · rate · currency · amount`

Use it for: Lets receipts and payments record (post-dated) cheques with bank, branch and due date, which is central to cash-flow follow-up.

Every commercial document, from order to journal entry

Voucher account lines (journal lines) · 11 fields · p. 14

One ledger line per account in a voucher: account, amount, debit/credit side, six cost centers and a note (pp. 14–15). Mandatory only in journal vouchers (p. 13). The manual gives no comments for this class.

`position · account_code · amount · debit_credit · costcenter1 · costcenter2 · costcenter3 · costcenter4 · costcenter5 · costcenter6 · note`

Use it for: Enables general-ledger journal entries with cost-center tagging to be posted into Shamel from outside tools.

Voucher insert result · 3 fields · p. 15

The answer to /Voucher/Post2, printed as class Result (p. 15): a success flag, a list of error messages and the number of the voucher that was created. A different class also named Result is returned by /CostCenter/Post.

`Success · ErrorMessage · refrenceId`

Use it for: Confirms a voucher was created and gives its Shamel number, needed for follow-up, linking and audit.

Voucher summary row · 9 fields · p. 18

A short, flat voucher row returned by /Voucher/GetExtraVouchers, printed as 'public class Voucher' (p. 18); the name ExtraVoucher is assigned here to tell it apart from the full Voucher. It has amount, direction and the customer's balance but no lines. What 'extra' means is not explained (ISCO-40b).

`type · code · date · customer_code · amount · currency · rate · cr_dr · customer_balance_at_voucher_date`

Use it for: A light list of customer-affecting documents with amounts and running balance, good for customer activity views and change detection.

Voucher list filters · 4 fields · p. 17

An unnamed JSON object sent in the request header 'filters' of /Voucher/GetVouchers and /Voucher/GetExtraVouchers (p. 17, repeated pp. 17–18); the name VoucherFilters is assigned here. There is no filter by voucher type and no paging (ISCO-40c, ISCO-21).

`customerCode · currencyCode · fromDate · toDate`

Use it for: Lets tools pull all vouchers for a period, optionally for one customer or currency, e.g. for daily sales or customer history.

09 REPORTS & COST CENTERS

Ready-made summaries computed by Shamel

Three reports come straight from Shamel: a customer account statement, the last prices of an item, and a cost-center statement. Cost centers can also be created.

Endpoints

Endpoint	What it returns	Inputs & filters	Sync
`/Report/GetLastPriceReport`	Last-prices report for an item: recent transaction dates, prices, quantities and customers.	ItemCode, barcode	
`/Report/GetAccountsheet`	Account statement (debits, credits, running balance) for a customer over a date range.	obj	
`/CostCenter/GetCostCenterAccountsheet`	Account statement for a range of cost centers (optionally one account) over a date range.	filters	

Records

Customer statement filters · 9 fields · p. 12

The filters of the customer account statement report (كشف حساب محاسبي, p. 3), printed as class AccountsheetFilters on p. 12 and sent as JSON in the form field 'obj' of /Report/GetAccountsheet. A different class with the same name is used by the cost-center statement. No field has a comment.

`customerCode · currencyCode · fromDate · toDate · showOtherCurrencies · showStocks · showCheqs · showNotPayedCheqs · showReturnedCheq`

Use it for: Drives the customer account statement, the key document for collections, customer reconciliation and sending statements to customers.

Cost-center statement filters · 8 fields · p. 19

The filters of the cost-center account statement report (تقرير كشف حساب لمركز تكلفة, p. 3), printed as class AccountsheetFilters on p. 19 and sent as JSON in the form field 'filters' (not 'obj') of /CostCenter/GetCostCenterAccountsheet. No field has a comment.

`fromCostcenter · toCostcenter · account · type · currencyCode · fromDate · toDate · showOtherCurrencies`

Use it for: Drives the cost-center statement, which shows income and costs per cost center (e.g. a project or department).

Statement line · 8 fields · p. 13

One row of an account statement, returned as a list by both the customer statement and the cost-center statement (printed identically on pp. 13 and 19). All amounts are strings with an unstated number format (ISCO-38b). No field has a comment.

`date · code · type · credit · debit · start_date · balance · note`

Use it for: The statement rows (date, document, debit, credit, balance) that a tool can show, export, send to a customer or analyse.

Cost center (new) · 6 fields · p. 18

A new cost center to insert, sent as JSON in the form field 'obj' of /CostCenter/Post (p. 18). The manual wrongly titles the endpoint 'Insert new customer' (copy-paste error; p. 2 lists 'مركز تكلفة جديد', new cost center). No field has a comment, and no endpoint lists existing cost centers (ISCO-39b).

`code · name · name_en · father · type · date`

Use it for: Lets an external tool create new cost centers (e.g. projects or departments) in Shamel so that vouchers can be tagged with them.

Cost-center insert result · 2 fields · p. 19

The answer to /CostCenter/Post, printed as class Result (p. 19): only a success flag and one error message. Unlike Result (voucher) and PostResponse, it returns no identifier.

`Success · ErrorMessage`

Use it for: Tells whether the cost center was created; since no id comes back, the code that was sent is the only reference.

10 WRITING DATA

Four ways to create records, none to change them

Writes go through four endpoints. Each takes the new record as a JSON string in the form field obj and answers with Success, an error message and, where documented, the new record's code. There is no update, cancel or delete, and the manual documents no protection against posting the same document twice.

Endpoint	What it creates	Returns
`/Customer/Post`	Insert a new customer; Shamel assigns and returns the new customer code.	PostResponse (JSON)
`/Item/Post`	Insert a new item with a caller-chosen code, one main unit and one price.	PostResponse (JSON)
`/Voucher/Post2`	Insert a new voucher of any of the 13 documented types (order, invoice, return, receipt, payment, notes, journal, deliveries, stock-out).	Result (voucher variant, JSON)
`/CostCenter/Post`	Insert a new cost center with a caller-chosen code.	Result (cost-center variant, JSON): no identifier returned

The 13 voucher types /Voucher/Post2 can create

Type	Name in the manual	Meaning	Arabic	Code prefix
0	`Order`	Sales order	طلبية	`O`
2	`PurchaseDelivery`	Purchase delivery (goods received)	ارسالية مشتريات	–
4	`ReturnSalesInvoice`	Sales return (goods returned by a customer)	مردودات مبيعات	`S`
5	`ReturnedPendingSalesDelivery`	Return from a consignment (on-sale) delivery	مردود من ارسالية برسم البيع	`F`
9	`SalesInvoice`	Sales invoice	فاتورة مبيعات	`I`
10	`SalesDelivery`	Sales delivery (goods dispatched for sale)	ارسالية مبيعات	`V`
12	`Stockout`	Goods issue (stock-out)	اخراج بضاعة	`B`
16	`InternalDelivery`	Internal delivery (transfer between stores, inferred)	ارسالية داخلية	`N`
22	`Credit`	Credit note	اشعار دائن	`C`
23	`Debit`	Debit note	اشعار مدين	`D`
24	`Receipt`	Receipt voucher (money received)	قبض	`R`
27	`Payment`	Payment voucher (money paid out)	صرف	`P`
28	`Journal`	Journal entry	قيد	`J`

What a voucher carries

A header (type, date, customer, currency and rate, salesman, store, discount, VAT, notes), item lines (item, unit, store, quantity, price, bonus, discount, cost centers), cheques and, for journals, account lines.

Codes

Leave the code empty and Shamel numbers the voucher; send one and it must start with the type's prefix letter (p. 13, 15–16). Purchase deliveries have no documented prefix.

Mistakes

Because only inserts are documented, a wrong document can only be corrected with an offsetting one (return, credit note, journal) or by hand in Shamel (our reading). Bawaba adds idempotency and an approval queue for this reason.

11 WHAT WE CAN BUILD

Tool ideas, checked against the API

Each idea lists the Shamel endpoints it needs and how feasible it is today, judged from the manual alone. “Milestone” is the Bawaba milestone that enables it (M1 reads, M2 writes and approvals, M3 webhooks).

6
ready: documented endpoints are enough

21
partial: works with caveats or an ISCO answer

3
blocked: needs something the API lacks

Sales & customers

Live sales dashboard M1 — PARTIAL

لوحة مبيعات مباشرة

Shows today's, this week's and this month's sales invoices and returns by customer, salesman, item and store, refreshed through the day, so the owner sees revenue without waiting for an accountant's report.

Voucher/GetVouchers filters only by customerCode, currencyCode, fromDate and toDate (p. 17), with no voucher-type, salesman or store filter and no paging, so each refresh downloads every voucher in the date range and keeps types 9 and 4 client-side; which Voucher class it returns and whether header balance is net of discount are not stated (pp. 13, 17). · ISCO-40, ISCO-21, ISCO-24, ISCO-25, ISCO-46, ISCO-26

`/Voucher/GetVouchers` · `/Customer/GetCustomers` · `/Item/GetItems` · `/Files/GetStores` · `/Files/GetSalesmen` · `/Files/GetCurrencies` · `/Files/GetCurrencyRates`

Customer 360 view M1 — READY

ملف الزبون المتكامل (360)

One screen per customer with contact details, assigned salesman, price class, credit limit, current balance, balances per currency, unpaid and returned cheque totals, classifications, GPS location and the serial-numbered goods they bought.

Every section uses documented headers and fields of GetCustomers (pp. 6–8), GetCustomerBalances (p. 8) and GetItemsSerials (pp. 11–12); the caveats are that fields such as attr, creditType and citizen_code are undocumented and the manual does not say whether balance already includes the cheque balances (p. 7). · ISCO-31, ISCO-32, ISCO-30, ISCO-25

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Item/GetItemsSerials` · `/Files/GetCities` · `/Files/GetPriceClasses`

Customer statement sender & portal M1 — PARTIAL

إرسال كشوف الحساب وبوابة الزبائن

Produces a customer's account statement for any period as a PDF or WhatsApp message, or lets a trusted customer open their own statement online, replacing manual statement requests to the accountant.

Report/GetAccountsheet is documented (pp. 12–13), but credit, debit and balance come back as strings in an unstated number format and start_date and type have no stated meaning, so totals must be parsed defensively; a customer-facing portal also relies on Bawaba scoping each token to one customer, because the API's only login is a Shamel user (p. 4). · ISCO-38, ISCO-24, ISCO-15

`/Report/GetAccountsheet` · `/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Files/GetCurrencies`

Order status tracker M1 — PARTIAL

متابعة حالة الطلبيات

Lets office staff (or customers through a WhatsApp bot) check whether a sales order is not ready, ready, closed, partly sent or sent, using the order's paper reference number.

GetStatus takes only Header[manual] (presumably the order's manual_code, inferred) and returns the 0–4 status (p. 16), so orders entered without a manual_code cannot be looked up, and the response format and the result for an unknown or shared manual code are undocumented; automatic 'your order is ready' messages need Bawaba's M3 order.status_changed events. · ISCO-42, ISCO-43

`/Voucher/GetStatus` · `/Voucher/GetVoucher`

Tool ideas, checked against the API

New-customer onboarding form M2 PARTIAL

نموذج تسجيل زبون جديد

A form for salesmen or office staff to register a new customer (name, VAT number, phones, address, city, salesman, level, price class) that first checks for an existing customer with the same VAT number and then creates the account in Shamel.

Customer/Post inserts a customer and returns the new code (pp. 8–9), but the write model has no currency, credit limit or discount, MobileNumber is commented 'فاكس' (fax), Town has no lookup list, a mistake cannot be corrected because there is no update endpoint, and whether a retried insert creates a duplicate is unknown. · ISCO-48, ISCO-32, ISCO-43, ISCO-26, ISCO-19

`/Customer/GetCustomers` · `/Files/GetCities` · `/Files/GetPriceClasses` · `/Files/GetSalesmen` · ✎ `/Customer/Post`

Field sales (salesmen)

Salesman order-entry app M2 PARTIAL

تطبيق إدخال الطلبيات للمندوبين

Mobile app where each salesman sees only their own customers, with balance, prices and stock, and enters sales orders and returns on the spot instead of on paper; selected orders can wait for owner approval in Bawaba before reaching Shamel.

Orders (Order = 0) and sales returns are documented writes (pp. 13–15) and GetCustomers is limited to the salesman's customers by default (p. 7), but orders cannot be edited or cancelled through the API, a retried post may duplicate (no idempotency is documented), and the required fields and whether Shamel checks stock or credit limits are not stated. · ISCO-43, ISCO-47, ISCO-25, ISCO-46, ISCO-44, ISCO-19

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Item/GetItems` · `/Item/GetItemsStores` · `/Customer/GetCustomerPricesAsRows` · `/Files/GetStores` · ✎ `/Voucher/Post2`

Collections (receipts) app M2 PARTIAL

تطبيق التحصيل (سندات القبض)

Lets salesmen and collectors record cash and cheque payments from customers in the field, with bank, branch, cheque number and due date, and gives the customer an instant receipt.

Receipt = 24 and the cheque lines are documented (pp. 14–15), but the manual does not say where the cash part of a receipt goes (balance, an accounts line or elsewhere) or whether Shamel computes the totals, and a duplicated receipt cannot be deleted through the API. · ISCO-47, ISCO-46, ISCO-43, ISCO-25

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Files/GetBanks` · `/Files/GetBanksBranchs` · `/Files/GetCurrencies` · `/Files/GetCurrencyRates` · ✎ `/Voucher/Post2`

Delivery & van-sales app M2 PARTIAL

تطبيق التوزيع والبيع من السيارة

Shows a salesman or driver the delivery (load) prepared for them, lets them invoice customers from that load and record returns, and reconciles what is left in the van.

GetDelivery returns one Voucher 'prepared to the salesman' with no inputs (p. 16), so the result with zero or several prepared deliveries is unknown; 'invoice from a delivery' is listed (p. 3) but has no voucher type of its own, and both using SalesInvoice with delivery_id/delivery_pos and reading car_code as a van store are inferences. · ISCO-41, ISCO-47, ISCO-27, ISCO-25, ISCO-43

`/Voucher/GetDelivery` · `/Files/GetStores` · `/Item/GetItemsStores` · `/Customer/GetCustomers` · ✎ `/Voucher/Post2`

Customer map & route planner M1 READY

خريطة الزبائن وتخطيط خطوط السير

Plots each salesman's customers on a map with their balance and stopped status, so supervisors can plan daily routes and see areas that are poorly covered.

latitude, longitude, city and salesman_code are documented customer fields and getAll returns every salesman's customers (p. 7); the caveats are that coordinates are strings that may be empty, town has no lookup list, and salesman names depend on the undocumented GetSalesmen (p. 6). · ISCO-26, ISCO-25, ISCO-32

`/Customer/GetCustomers` · `/Files/GetCities` · `/Files/GetSalesmen`

Tool ideas, checked against the API

Field updates to customer records M2

BLOCKED

تحديث بيانات الزبائن من الميدان

Would let salesmen correct a customer's phone or address, capture the shop's GPS location, or mark a customer as stopped, directly from their phone.

There is no customer update endpoint: Customer/Post only inserts new customers (p. 8) and its write model has no latitude, longitude or stopped field (p. 9), so corrections and GPS captures can only be queued in the tool for office staff to enter in Shamel. · ISCO-52, ISCO-55

`/Customer/GetCustomers`

Stock & items

Multi-store stock view M1

READY

عرض المخزون في جميع المستودعات

Shows how much of each item is on hand in every store and van, with shelf locations, so sales and warehouse staff can answer 'where do we have it?' instantly.

GetItemsStores and getItemDetails return quantity per store with shelf and main unit (pp. 10–11) and GetStores names the stores (p. 6); the open points are only about interpretation: that qnty is in the main unit, whether it nets off open orders, and what store type and car_code mean. · ISCO-36, ISCO-27, ISCO-25

`/Item/GetItemsStores` · `/Item/getItemDetails` · `/Item/GetItems` · `/Files/GetStores`

Low-stock alerts & reorder suggestions M1

PARTIAL

تنبيهات نقص المخزون واقتراحات إعادة الطلب

Watches stock levels against minimum quantities the company sets per item and store, alerts the purchasing person by email or WhatsApp, and suggests reorder quantities from recent sales.

Quantities are documented (pp. 10–11), but the item model has no minimum or reorder level (pp. 9–10), so thresholds must live in the tool, and the lastupdatedate header of GetItemsStores is listed without any explanation (p. 10), so near-real-time alerts mean re-reading the whole stock list until Bawaba's M3 stock.changed events take over. · ISCO-22, ISCO-36, ISCO-21, ISCO-20, ISCO-40

`/Item/GetItemsStores` · `/Item/GetItems` · `/Voucher/GetVouchers` · `/Files/GetStores`

Price lookup tool M1

PARTIAL

أداة الاستعلام عن الأسعار

Answers 'what is the price of this item for this customer?' by combining the six price-class prices, the customer's price class, customer-specific special prices and discounts, and the last prices at which the item was sold.

All the data is readable, but the link between price class N and sale_pariceN is inferred and unit_price_1..6 and discount_type are undocumented (pp. 6, 9–10, 16–17); GetCustomerPricesAsRows has no documented inputs, so every lookup downloads the whole price table, and GetLastPriceReport has no customer filter (p. 12). · ISCO-29, ISCO-33, ISCO-37, ISCO-15

`/Item/GetItems` · `/Files/GetPriceClasses` · `/Customer/GetCustomers` · `/Customer/GetCustomerPricesAsRows` · `/Report/GetLastPriceReport` · `/Files/GetCurrencies`

Serial & warranty lookup M1

READY

الاستعلام عن الأرقام التسلسلية والكفالة

Lets service staff type or scan a serial number and see which item it is, which store holds it and when its warranty starts and ends, or list all serial-numbered goods sold to a customer.

GetItemsSerials is documented with customer and serial filters and warranty start and end dates (pp. 11–12); only the status values are undocumented, and the response has no customer field, so the buyer is known only when searching by customer. · ISCO-49, ISCO-15, ISCO-35

`/Item/GetItemsSerials` · `/Item/GetItems`

Tool ideas, checked against the API

Warehouse app: receiving, transfers, goods issue and counts M2

PARTIAL

تطبيق المستودع: الاستلام والتحويل والإخراج والجرد

Handheld app for storekeepers to record goods received from suppliers, transfers between stores and vans, and goods issued, and to count stock and compare it with the book quantity.

The three voucher types are in VoucherTypes (p. 15), but PurchaseDelivery has no code prefix letter (pp. 15–16), the manual does not say how an internal delivery names its destination store or which fields each type requires, and none of the 13 documented types is a stock-in or adjustment (p. 15), so a count can post only shortages (as type 12). · ISCO-47, ISCO-45, ISCO-43, ISCO-36, ISCO-49

`/Item/GetItemsStores` · `/Item/GetItems` · `/Files/GetStores` · ✎ `/Voucher/Post2`

New items from a supplier price sheet M2

PARTIAL

إنشاء أصناف جديدة من قائمة أسعار المورّد

Reads a supplier's Excel or PDF price list, matches its lines to existing items by barcode or factory code, and creates the missing items in Shamel with name, main unit, price, group and barcode.

Item/Post inserts an item (pp. 19–20) but takes only the main unit and one price (price1); trademark, producing_company, style and model are numbers with no lookup list, item type values and the content of PostResponse.value are not stated, and whether a duplicate code is rejected is unknown. · ISCO-48, ISCO-43, ISCO-49, ISCO-19

`/Item/GetItems` · `/Item/GetItemsGroups` · `/Files/GetCurrencies` · ✎ `/Item/Post`

Bulk price & item updates M2

BLOCKED

تحديث الأسعار وبيانات الأصناف بالجملة

Would let the company raise prices by a percentage, apply a supplier's new costs, or fix names and barcodes for many items at once from a spreadsheet.

The API has no update endpoint for items or anything else: Item/Post is documented only as 'Insert new Item' (pp. 19–20), so existing prices and item details can only be changed in the Shamel desktop program, and a tool can at most prepare a change list for staff to key in. · ISCO-52, ISCO-55

`/Item/GetItems`

Finance & accounting

Collections & credit control board M1

PARTIAL

لوحة التحصيل ومراقبة السقوف الائتمانية

Ranks customers by what they owe, flags those over their credit limit or with returned cheques, and estimates how old each debt is, so collectors and salesmen know whom to call first.

Balances and credit limits are documented (pp. 7–8), but there is no aging or overdue report and vouchers carry no due date (only the customer's paymentTermsCode, which has no lookup list), so 'overdue' must be estimated from statement rows whose amounts are strings (p. 13), and whether balance includes cheques is not stated. · ISCO-31, ISCO-38, ISCO-40, ISCO-49

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Report/GetAccountsheet` · `/Voucher/GetExtraVouchers`

Cheque tracker M1

PARTIAL

متابعة الشيكات

Lists post-dated cheques received from customers by due date, bank and branch, and shows each customer's totals of not-yet-paid and returned (bounced) cheques, so finance can plan deposits and chase bounced cheques.

Per-customer totals of unpaid and returned cheques are documented (p. 7) and individual cheques with due dates appear inside vouchers (p. 14), but no endpoint lists cheques or gives a cheque's status (deposited, cleared, returned), so cheques must be collected by scanning GetVouchers by date (p. 17) and their status inferred from statements. · ISCO-31, ISCO-38, ISCO-40, ISCO-21

`/Customer/GetCustomers` · `/Voucher/GetVouchers` · `/Voucher/GetVoucher` · `/Report/GetAccountsheet` · `/Files/GetBanks` · `/Files/GetBanksBranchs`

Tool ideas, checked against the API

Currency rate board M1 — READY

لوحة أسعار صرف العملات

Shows the currencies defined in Shamel (the manual's examples are "NIS" and "$") with the latest exchange rates, and keeps its own history so quotes and reports convert amounts consistently.

Both endpoints are documented (pp. 4–5); GetCurrencyRates returns only the 'latest 10' rates with no date or currency filter, so a longer history must be recorded by the tool itself, and is_base is not explained. · ISCO-28

`/Files/GetCurrencies` · `/Files/GetCurrencyRates`

Automated journal entries & credit/debit notes M2 — PARTIAL

قيود يومية وإشعارات دائنة ومدينة آلية

Posts recurring or bulk accounting entries (bank charges, salaries, allocations) and customer credit or debit notes from a spreadsheet or another system, entered as unposted so an accountant reviews them in Shamel.

Journal vouchers are supported and accounts is mandatory for them (p. 13), but debit_credit values are undocumented, no endpoint lists GL accounts or cost centers, and a wrong entry cannot be edited or deleted through the API (all writes are inserts), so account codes must come from the accountant and every post needs a reversal plan. · ISCO-49, ISCO-47, ISCO-46, ISCO-43, ISCO-44, ISCO-39

`/Customer/GetCustomers` · `/Files/GetCurrencies` · `/Files/GetCurrencyRates` · `/Voucher/GetVoucher` · ✎ `/Voucher/Post2` · ✎ `/CostCenter/Post`

Cost-center reporting M1 — PARTIAL

تقارير مراكز التكلفة

Reports movements per cost center (for example a branch, vehicle or project) for any period, so the owner sees which part of the business makes or loses money.

The cost-center statement is documented (p. 19), but no endpoint lists cost centers, the int type filter and the account filter are unexplained, and amounts are strings, so the tool needs cost-center codes from the accountant and defensive number parsing; creating new cost centers (/CostCenter/Post) is a separate M2 write that returns no identifier (pp. 18–19). · ISCO-39, ISCO-38, ISCO-49, ISCO-24

`/CostCenter/GetCostCenterAccountsheet`

Financial statements, cash position & supplier payables M1 — BLOCKED

القوائم المالية والسيولة وذمم الموردين

Would show profit and loss, a trial balance, cash and bank balances and what the company owes each supplier, directly from Shamel.

The manual offers only three reports, the customer account statement, last prices and the cost-center statement (p. 3), with no chart of accounts, trial balance, P&L, bank balance or supplier list, and no purchase-invoice type among the documented VoucherTypes (p. 15); GetAccountsheet's customerCode might accept other account codes, but that is unconfirmed (p. 12). · ISCO-38, ISCO-52, ISCO-49, ISCO-55

`/Report/GetAccountsheet`

E-commerce & integrations

Website catalogue & stock sync M1 — PARTIAL

مزامنة الكتالوج والمخزون مع الموقع الإلكتروني

Publishes items with names, groups, prices, images and availability to the company website or a marketplace and keeps them in step with Shamel automatically, while Bawaba hides cost prices and GL accounts.

Items, groups and stock are documented, but lastupdatedate is a date-only 'updated after' filter with no deletion signal (pp. 4, 9), the image_path format is undocumented and no endpoint returns image bytes (p. 11), so the sync must re-read with overlap and images stay off until ISCO explains them; M3 events later replace the polling. · ISCO-22, ISCO-23, ISCO-34, ISCO-29, ISCO-21

`/Item/GetItems` · `/Item/GetItemsGroups` · `/Item/GetItemsCatalog` · `/Item/GetItemsStores` · `/Files/GetPriceClasses`

Online orders into Shamel (web-shop & B2B portal) M2 — PARTIAL

إدخال الطلبات الإلكترونية إلى الشامل (متجر إلكتروني وبوابة تجّار)

Turns orders from the company's web-shop, or from a portal where retail shop owners log in to see their own prices and balance, into Shamel sales orders, carrying the web order number so each order can be traced.

Order insert is documented (pp. 13–15) and manual_code can carry the web reference (p. 13), but there is no idempotency, so a retried web order may duplicate, the purpose of system_id ('value must be between 1000 and 2000', p. 14) is unknown, status can be read back only by manual code (p. 16), and anonymous web buyers must be mapped to a Shamel customer account. · ISCO-43, ISCO-44, ISCO-42, ISCO-33, ISCO-47

`/Customer/GetCustomers` · `/Customer/GetCustomerPricesAsRows` · `/Item/GetItems` · `/Item/GetItemsStores` · `/Voucher/GetStatus` · `/Report/GetAccountsheet` · ✎ `/Voucher/Post2` · ✎ `/Customer/Post`

Tool ideas, checked against the API

Event-driven notifications & integrations M3

PARTIAL

إشعارات وتكاملات فورية مبنية على الأحداث

Sends WhatsApp, email or Slack messages and pushes data to other systems when something changes: an order becomes ready, an item's stock drops, a customer's balance changes, or a new invoice or customer appears.

Shamel documents no push or webhook mechanism, so Bawaba's M3 events come from polling every 5 minutes and comparing results; lastupdatedate is date-only and exists on only three endpoints (pp. 7, 9, 10), deletions are never signalled, and GetVouchers filters only by a date range whose basis is unstated (p. 17), so events arrive with a delay. · ISCO-22, ISCO-23, ISCO-42, ISCO-20, ISCO-24

`/Voucher/GetStatus` · `/Item/GetItemsStores` · `/Customer/GetCustomerBalances` · `/Customer/GetCustomers` · `/Item/GetItems` · `/Voucher/GetVouchers`

Management & BI

Management KPI dashboard M1

PARTIAL

لوحة مؤشرات الأداء للإدارة

One page for the owner with sales against last month, collections, total receivables, returned cheques, stock value by store and an estimated gross margin by item group.

Receivables and stock quantities are documented (pp. 7–11), but sales and collections depend on scanning GetVouchers without a type filter (p. 17), stock value and margin depend on purchase_price, whose meaning (last cost or average cost) is not stated (pp. 9–10), and multi-currency totals rely on only the latest 10 rates (p. 5). · ISCO-40, ISCO-29, ISCO-46, ISCO-28, ISCO-21

`/Voucher/GetVouchers` · `/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Item/GetItemsStores` · `/Item/GetItems` · `/Files/GetCurrencyRates`

BI export to Excel / Power BI M1

PARTIAL

تصدير البيانات إلى Excel وPower BI

Nightly export of customers, items, stock, balances and vouchers into Excel, Google Sheets, Power BI or a small data warehouse, so analysts can build their own reports without touching Shamel.

Master data exports are straightforward, but no endpoint documents paging, totals or row limits, voucher reads are limited to date ranges with no type filter (p. 17), and edits or deletions of vouchers are not signalled, so the warehouse must re-scan recent periods every night, and response sizes for long ranges are unknown. · ISCO-21, ISCO-23, ISCO-40, ISCO-20, ISCO-17

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Item/GetItems` · `/Item/GetItemsGroups` · `/Item/GetItemsStores` · `/Voucher/GetVouchers` · `/Files/GetStores` · `/Files/GetCurrencies` · `/Files/GetCities`

Salesman performance & commission M1

PARTIAL

أداء المندوبين وعمولاتهم

Ranks salesmen by sales, returns, collections and new customers each month and calculates commissions with the company's own rules.

VoucherHeader.salesman is documented (p. 13), but GetVouchers cannot filter by salesman or type (p. 17), the response of GetSalesmen is not documented at all (p. 6), and whether vouchers are salesman-scoped for a salesman user is unknown; new customers can only be found by comparing customer lists over time, because no creation date is returned. · ISCO-26, ISCO-40, ISCO-25, ISCO-21

`/Voucher/GetVouchers` · `/Files/GetSalesmen` · `/Customer/GetCustomers`

AI assistant for customers & stock (read-only) M1

READY

مساعد ذكي للاستعلام عن الزبائن والمخزون (قراءة فقط)

A chat assistant (for example in WhatsApp or Teams) that answers staff questions such as 'what does this customer owe?', 'how many of this item are in the main store?' or 'is this serial still under warranty?' using a read-only Bawaba token.

Customer, balance, item, stock and serial questions map to documented lookups by code (GetCustomers code, getItemDetails itemCode, GetItemsSerials serial; pp. 6–12); questions about sales history inherit the GetVouchers limits (p. 17), and search by name is done by Bawaba over a full download because Shamel has no name-search parameter. · ISCO-15, ISCO-31, ISCO-36, ISCO-25

`/Customer/GetCustomers` · `/Customer/GetCustomerBalances` · `/Item/GetItems` · `/Item/getItemDetails` · `/Item/GetItemsStores` · `/Item/GetItemsSerials` · `/Files/GetStores` · `/Files/GetPriceClasses`

12 LIMITS & GAPS

What the API does not do, and why it matters

Ordered by severity. Each entry says what it means for the tools we build and whether the Bawaba gateway can work around it.

high medium low

One all-powerful login; no permissions beyond salesman scoping · security (ISCO-04, ISCO-25)

Every operation needs a Shamel user number, password and a token from the login call ('الامان حيث تتطلب اي عملية ثلاث عناصر رئيسية: رقم المستخدم، كلمة المرور، Token' = 'security: any operation requires three main elements: user number, password, token', p. 2); after login each request carries only the username and token headers (p. 4). The manual describes no API keys, read-only users, per-endpoint rights or per-voucher-type rights; the only restriction it mentions is salesman scoping on GetCustomers (p. 7) and GetDelivery (p. 16). **Impact:** Any tool holding a Shamel login can read everything that login sees and insert any voucher type (orders, invoices, receipts, journals). Shamel itself cannot give a tool narrower access, and sharing one login means a single leak exposes the whole company.

Bawaba: This is Bawaba's main purpose: tools never see Shamel credentials. The owner issues unlimited Bawaba tokens, each with per-endpoint scopes (e.g. customers:read), per-voucher-type write scopes, data scopes (stores, salesmen, customers, currencies, price classes), field hiding (e.g. purchase_price, balances, credit limits), IP allowlists, optional expiry and instant revoke; writes can be held for owner approval (00 §5-6, 03 §2). Residual: the Shamel service account Bawaba uses is still all-powerful inside Shamel; a least-privilege Shamel account depends on ISCO.

No HTTPS documented; password and token may travel in plain text · security (ISCO-05, ISCO-06, ISCO-07, ISCO-08)

The manual gives no base URL, scheme, port or TLS information and never mentions HTTPS (pp. 1-4). The password is sent as a form field to /Main/Login and the token as a plain header on every request (p. 4). **Impact:** Unless ISCO confirms HTTPS, anyone who can watch the network between a caller and Shamel can capture the password or token. Shamel must never be exposed directly to the internet, and tools should not call it from outside the office.

Bawaba: The Bawaba Connector runs next to Shamel (ideally on the same server) and opens an outbound encrypted tunnel (WebSocket with inner mutual TLS 1.3), so Shamel needs no public IP or open port and tools only reach Bawaba over TLS behind Cloudflare (00 §4, 01 §8). Direct mode refuses plain HTTP over the internet unless it runs over WireGuard or pinned HTTPS (02 DU-3). The short hop from Connector to Shamel stays plain HTTP unless ISCO supports HTTPS.

Every error is HTTP 500 with raw details · errors (ISCO-18)

'any error occurred the service will return Http error 500 with the error details' (p. 4). No error format, error codes or examples are given, so an expired token, a bad parameter, a missing record and a server crash all look the same at HTTP level. **Impact:** Tools cannot tell whether to log in again, fix the request, retry later or give up, and the raw details may expose internal information (exception text, SQL, server paths) to whoever sees the response.

Bawaba: An outcome classifier sorts every upstream reply (auth failure via a calibrated signature, business rejection, server error, timeout); clients get proper status codes (400/401/403/404/422/502/503/504) and a stable bilingual error catalogue with a request id. Shamel's raw text goes only to the encrypted audit log, never to clients (01 §7.5, 03 §22).

No pagination: lists return everything at once · performance (ISCO-21, ISCO-20)

List endpoints return all records in one response: 'Get all customers' (p. 7), 'Get all items' (p. 9), 'Get all items quantities' (p. 10), 'Get all items image' and 'Get all items with serials only' (p. 11). There are no page, limit, sort or total parameters; the only fixed limit is GetCurrencyRates' 'latest 10' (p. 5). GetVouchers returns full vouchers for the whole date range asked (p. 17). **Impact:** A dashboard showing one page of items or customers still makes Shamel build the full list; large tables can be many megabytes, slow and prone to timeouts; tools would have to hold and page the data themselves.

Bawaba: Snapshot pagination: page 1 makes one live call, later pages reuse that result for about 60 s through sealed cursors, with totals, sorting and Arabic-aware search; identical concurrent requests share one upstream call; voucher ranges are split into 7-day slices and capped at 31 days by default; response-size caps and warnings protect Shamel (00 §8, 03 §5, §12.2). Shamel still does the full work for every page 1.

No change feed for vouchers, balances or deletions · sync (ISCO-22, ISCO-23, ISCO-52)

The manual has no webhooks, events or change log. Vouchers can only be read by date range (p. 17), customer balances only as point-in-time values (pp. 7-8), and no endpoint reports deleted records. VoucherHeader.update_date exists but has no description and cannot be filtered on (p. 13). **Impact:** Tools must poll. Edits to back-dated vouchers, balance movements and records deleted in the desktop program go unnoticed unless a tool keeps re-scanning; real-time alerts are impossible from Shamel alone.

Bawaba: Webhooks (milestone 3) detect changes by polling every 5 minutes: tiered voucher scans (last 3 days every cycle, 35-day hourly sweep via GetExtraVouchers, nightly 35-day deep scan), balance and stock snapshot diffs, and *.removed after a record is missing from two full fetches (03 §17, 05 §2.4). Limits: up to about 5 minutes delay, changes to vouchers older than about 42 days are not detected, and a deletion looks the same as a record leaving the token's scope.

What the API does not do, and why it matters

No update, cancel or delete operations · writes (ISCO-52, ISCO-23, ISCO-55)

The capability list offers only data pulls, inserts and three reports (pp. 2-3), and every write endpoint is an insert: 'Insert new customer' (p. 8), 'Insert new voucher' (p. 13), 'Insert new Item' (p. 19) and /CostCenter/Post (p. 18). No endpoint edits, cancels, voids or deletes anything. **Impact:** Tools cannot change a customer's phone or credit limit, update item prices, cancel an order or void an invoice. Every change to existing data must be done by a person in the Shamel desktop program.

Bawaba: Cannot be fixed by a gateway — needs ISCO. Bawaba exposes no PATCH/DELETE for Shamel records and reduces the need for corrections through validation, the approval queue and sandbox testing (03 §4.3, §15).

Mistakes can only be fixed with offsetting documents · writes (ISCO-46, ISCO-47, ISCO-51)

Because writes are insert-only (pp. 8, 13, 18, 19), a wrong invoice can only be offset by another document such as a sales return (ReturnSalesInvoice=4) or credit note (Credit=22) (p. 15), or corrected by hand in Shamel. VoucherHeader.is_posted chooses whether the voucher is entered as posted ('هل يتم ادخال الحركة حالة مرحلة ام لا' = 'is the voucher entered as posted or not', p. 13), but what an unposted voucher does in Shamel is not documented. **Impact:** Every write tool needs a review step before sending; a buggy tool can fill the books with wrong entries plus their reversals, which accountants must then clean up.

Bawaba: Approval queue: writes from selected tokens (the default for agent-requested tokens) wait for the owner's approval on the server; every write is audited with its full payload and Shamel reference id; strict validation and a journal balance check; a sandbox with a mock Shamel; writes can be sent with is_posted=false for accountant review once ISCO confirms its meaning (00 §6, 03 §15.3, §15.7).

No idempotency: a retry can create a duplicate · writes (ISCO-43, ISCO-45, ISCO-54)

The manual documents no idempotency key and no duplicate check. VoucherHeader.code is optional and Shamel assigns a number if it is not sent ('اذا لم يتم ارسال رقم الحركة يتم اعطائها رقم من الشامل' = 'if the voucher number is not sent, Shamel gives it a number', p. 13); whether a repeated code or manual_code is rejected, and whether an HTTP 500 or timeout means nothing was saved, is not stated (pp. 4, 13-15). **Impact:** If the connection drops after a voucher is sent, retrying may create a second invoice or receipt: a real accounting error that cannot be deleted through the API. Tools must never blindly retry writes.

Bawaba: Every /v1 write requires an Idempotency-Key; a repeat with the same key replays the stored result instead of calling Shamel again. Writes are never retried once they may have reached Shamel; an uncertain outcome becomes 504 write_outcome_unknown for owner reconciliation, helped by a unique reference in manual_code (03 §15.1-15.2). Guarantees on the Shamel side still need ISCO.

Unknown rate limits and impact on desktop users · performance (ISCO-20)

Nowhere in the manual (pp. 1-20) are rate or concurrency limits, request timeouts, or the effect of heavy calls (full item, customer or stock lists; wide voucher ranges) on people using the Shamel desktop program mentioned. **Impact:** A buggy or over-eager dashboard polling full lists every few seconds could slow the accounting office or bring the service down, with no warning from Shamel.

Bawaba: A global cap on concurrent Shamel calls (default 8) with a fair queue per token, priority for interactive reads, at most 50% of slots for large calls and 25% for background jobs; per-token rate limits that also count upstream cost; coalescing of identical requests; timeouts and a circuit breaker; Bawaba-Warning headers (polling_too_frequent, use_webhooks) and owner e-mail alerts about abusive tools (01 §7.6, 03 §23). The right cap needs ISCO figures or a benchmark.

No lookup lists for many codes · data access (ISCO-26, ISCO-39, ISCO-48, ISCO-52)

The API uses codes it cannot list: GL accounts (credit_account, debit_account, p. 10; VoucherAccounts account_code, p. 14), cost centers (insert only, p. 18; costcenter1-6 on lines and accounts, pp. 14-15), towns (town p. 7, Town p. 9), payment terms (paymentTermsCode, p. 7), customer classifications and contact types (pp. 7-8), item types, trademarks, producing companies, styles and models (p. 20), organizations (orgnization_code) and payment types (payment_type, p. 14). Salesmen have an endpoint but no documented response (p. 6). **Impact:** Tools cannot offer drop-downs or validate these codes; journal-entry, cost-center and item-creation tools need codes typed by hand, and cost-center reports require the user to already know the codes.

Bawaba: Cannot be fixed by a gateway — needs ISCO. Bawaba pre-checks the codes it can list (customer, currency, store, city, bank, branch) and leaves the rest for Shamel to accept or reject (03 §15.7).

Only three reports; little history · data access (ISCO-28, ISCO-38, ISCO-52)

The API offers exactly three reports: account statement ('كشف حساب محاسبي' = 'accounting account statement'), last-price report and cost-center statement (p. 3; pp. 12-13, 19). There is no trial balance, profit and loss, aging, sales analysis or stock valuation. GetCurrencyRates returns only the 'latest 10' rates with no date filter (p. 5); customer balances come without history except the balance at a voucher's date (pp. 7-8, 17-18). **Impact:** Financial dashboards (profit, aging, sales by item or salesman) must be computed from raw vouchers, which is heavy without pagination, may miss voucher types, and cannot produce GL-based statements because accounts cannot be listed; historical exchange rates are not available.

Bawaba: Partly: snapshot paging, voucher range slicing, voucher summaries and combined endpoints (customer 360, stock across stores) make computing from raw data practical. Real financial reports need ISCO (03 §12-14).

What the API does not do, and why it matters

Salesman scoping can be switched off by the caller · security (ISCO-25)

GetCustomers returns only the logged-in salesman's customers, but the manual says 'To get all customers even not the salesman's customers fill Header[getAll]=true' (p. 7). It does not say whether getAll needs a permission, or which other endpoints (balances, prices, items, vouchers, reports) are salesman-scoped at all. **Impact:** A salesman's own Shamel login is not a safe sandbox for a tool: any caller holding its token can add getAll=true. Tools that must see only one salesman's data need that restriction enforced outside Shamel.

Bawaba: getAll is not a client parameter on /v1; Bawaba sends it only when the token's policy allows it and the identity is not a pinned salesman user (01 §7.2, 03 §2.6). Bawaba's own data scopes (salesmen, customers, stores) apply on top of Shamel's scoping.

Unknown token lifetime and session behaviour; no logout · security (ISCO-01, ISCO-02, ISCO-03, ISCO-54)

The manual only says 'Keep the token to send it with every request' (p. 4). It documents no expiry, refresh, logout or revocation, does not say whether a second login (API or desktop) ends the first session, and does not say what a wrong password or disabled user returns (AuthResult has success and userIsDisabled, p. 4). **Impact:** Tools that each log in as the same user may knock each other's sessions out; any call can fail with an expired token at any moment; a leaked token cannot be reliably killed; a wrong password retried in a loop could lock the account if Shamel has a lockout policy.

Bawaba: Bawaba holds one shared session per Shamel identity, logs in one at a time cluster-wide, uses a single session holder by default, re-logs in after an auth failure and retries reads once (never writes), and backs off 30 s to 5 min after failed logins with an owner alert (01 §7.1); a calibration command tests whether two sessions can coexist. Revoking an already-issued Shamel token cannot be done by a gateway: needs ISCO (logout or revocation).

Write results come in three shapes with free-text errors · errors (ISCO-19, ISCO-39)

/Customer/Post and /Item/Post return PostResponse {Success, ErrorMessage (string), value} (pp. 9, 20); /Voucher/Post2 returns Result {Success, ErrorMessage (string[]), refrenceId} (p. 15); /CostCenter/Post returns Result {Success, ErrorMessage (string)} with no id at all (p. 19). Failures can also arrive as HTTP 500 (p. 4). No list of validation messages or error codes is given. **Impact:** Each write needs its own handling; tools cannot reliably show which field was wrong or why Shamel refused a voucher (unknown customer, no stock, closed period...). The created cost center's code can only be assumed to be the one that was sent.

Bawaba: Bawaba normalises all three shapes, maps known Shamel messages to stable codes (duplicate_code, reference_not_found, validation_failed) and returns everything else as 422 upstream_rejected. Before sending it validates the body and pre-checks customer, currency, store, city and bank codes against live data, so most mistakes come back as field-level errors (03 §15.7, §22.3; writes are milestone 2).

Parameters go in HTTP headers (proxy, size and Arabic-text problems) · data access (ISCO-11, ISCO-13, ISCO-14)

Apart from login and the obj/filters form posts, every input is a custom HTTP header: username and token (p. 4); code, vatreg, getBalance, getAll (p. 7); serial, itemCode, store, model (p. 9); voucher_code, a name with an underscore (p. 17); and a whole JSON object in the filters header (p. 17). The same item-code header is spelled itemCode, itemcode and ItemCode (pp. 9, 10, 12). **Impact:** Many reverse proxies (nginx by default) drop headers with underscores, so GetVoucher can silently fail behind one; JSON in a header hits size limits and lands in access logs; Arabic or other non-ASCII codes cannot safely be sent in headers; browsers, no-code tools and caches handle header-driven APIs poorly.

Bawaba: Tools use a clean /v1 REST API with query parameters, path ids and JSON bodies. Bawaba builds Shamel headers from a fixed table with the exact printed spellings, ASCII-escapes JSON, and calls Shamel directly from the Connector so no proxy strips voucher_code. Codes with Arabic or other non-ASCII characters are looked up by fetching the list and matching exactly, which costs more and is flagged with a warning (01 §7.2, 03 §4.2).

Writes are JSON packed inside a form field · writes (ISCO-12, ISCO-16, ISCO-17)

Writes and the account-statement report take a 'post parameter with key name [obj]' whose value 'must be in Json format' (pp. 8-9, 12, 13, 18, 19); the cost-center report uses [filters] instead (p. 19). Only /Main/Login states x-www-form-urlencoded (p. 4); the manual shows C# classes only, never a JSON sample, and property names mix styles (Name, VatReg next to customer_code). **Impact:** Every write must be encoded twice (JSON, then form encoding). Whether misspelled or unknown property names are rejected or silently ignored is unknown, so a small mistake could create a wrong record that the API cannot later correct.

Bawaba: Tools send ordinary JSON bodies to /v1, checked against a strict schema (unknown fields rejected, decimals exact). Bawaba maps them to typed Shamel classes with the exact printed property names and does the form encoding itself (01 §7.3, 03 §15).

No voucher-type filter and no voucher lookup by manual_code · data access (ISCO-40, ISCO-42, ISCO-43, ISCO-52)

GetVouchers and GetExtraVouchers filter only by customerCode, currencyCode, fromDate and toDate (p. 17). GetVoucher needs the voucher code in the voucher_code header (p. 17). The only call that takes the paper reference (manual_code) is GetStatus ('Fill Header[manual]'), and it returns only an order status 0-4 (p. 16). **Impact:** 'All receipts this month' means downloading every voucher in the range and filtering locally. A tool that stored its own reference in manual_code cannot fetch that voucher directly, and orders without a manual_code cannot have their status checked.

Bawaba: /v1/vouchers adds type, manual_code, salesman, store and posted filters plus search, evaluated on the snapshot (same upstream cost as the unfiltered range). /v1/order-status also accepts an order_code (GetVoucher, then GetStatus with its manual_code). A real server-side filter or lookup needs ISCO (03 §12.2, §12.5).

What the API does not do, and why it matters

Few server-side filters; no search or sort · data access (ISCO-15, ISCO-33, ISCO-36, ISCO-52)

GetItemsStores filters only by lastupdatedate and itemcode, not by store (p. 10); Customer/GetCustomerPricesAsRows documents no inputs at all (p. 16); GetItemsGroups and GetItemsCatalog take none (p. 11); no endpoint searches by name or sorts. Existing filters match on codes, and whether matching is exact, prefix or partial is not stated (pp. 7-11). **Impact:** Common questions ('stock in store 3', 'customers named النور', 'special prices for this customer') need a full download and local filtering, costing Shamel a full list every time.

Bawaba: Bawaba adds filters, sorting and Arabic-normalised search (alef/hamza forms, taa marbuta, diacritics) over the snapshot, plus combined endpoints such as customer 360 and stock across stores; the docs mark which filters really reduce Shamel's work ('pushdowns') so AI-built tools prefer them (03 §5.4-5.6, §14). The upstream cost of unfilterable lists remains.

lastupdatedate is date-only and exists on only three endpoints · sync (ISCO-22, ISCO-24)

'Date format is always yyyy-MM-dd' (p. 4). GetCustomers and GetItems return records 'updated after lastupdatedate' (pp. 7, 9), and GetItemsStores lists the header without explaining it (p. 10); no other endpoint has it. Time of day, timezone, whether 'after' includes the same day, and what counts as an update are not stated, and records carry no last-modified field (pp. 7, 9-10). **Impact:** Incremental sync is coarse: a tool can miss changes made earlier the same day, or must re-download one to two days of changes each time; vouchers, balances, prices, serials and master data have no incremental option at all.

Bawaba: /v1 offers updated_since on customers, items and stock, sending Shamel a safely earlier date (one day plus a 6-hour overlap) and documenting a 'superset' guarantee (extra records possible, none missed); the webhook detector hashes records so repeats create no events, and runs a daily full reconciliation (03 §5.7, 05 §2.4.1). Exact timestamps need ISCO.

Undocumented or half-documented endpoints · documentation (ISCO-12, ISCO-26, ISCO-33, ISCO-40, ISCO-41, ISCO-42)

/Files/GetSalesmen has only a heading: no inputs, no response (p. 6). Customer/GetCustomerPricesAsRows has no inputs or description, only a field list (pp. 16-17). GetExtraVouchers' purpose ('extra') is not explained (pp. 17-18); GetDelivery's behaviour with zero or several deliveries and GetStatus's response format are not given (p. 16). An HTTP method is implied ('post parameter') for only 7 of the 30 endpoints. **Impact:** Tools cannot rely on the salesmen list, customer special prices or voucher summaries until their real shape is captured; building on guesses risks breakage when the true behaviour is seen.

Bawaba: These are exposed as provisional: a tolerant decoder accepts several plausible shapes (e.g. GetSalesmen, GetStatus, GetDelivery), unknown fields are counted as schema drift, and the method per endpoint is configurable. Final shapes come from a live capture or from ISCO (03 §8.8, §12.3-12.5).

Coded fields and enums without values · documentation (ISCO-44, ISCO-45, ISCO-47, ISCO-49)

Many numeric codes have no list of meanings: Store type (p. 6); Customer creditType, attr, trans_permissions, level (p. 7); CustomerContact type (p. 8); Item type, eval_unit and the stopped/paused/has_* flags (p. 10); ItemSerial status (p. 12); payment_type and VoucherItems unit (p. 14); VoucherAccounts debit_credit (p. 15); discount_type (p. 16); cr_dr (p. 18); statement type (pp. 13, 19) and cost-center type (pp. 18-19). VoucherTypes skips many numbers (p. 15), PurchaseDelivery has no code prefix letter (pp. 15-16), and system_id is only 'value must be between 1000 and 2000' (p. 14). **Impact:** Tools can show raw numbers but not labels; journal entries cannot be written safely until debit_credit is known; purchase deliveries cannot be given their own codes.

Bawaba: Bawaba publishes these as raw integer ids marked provisional and adds named values once known; journal writes stay unavailable until the debit_credit mapping is configured; purchase_delivery codes are left to Shamel; system_id is a fixed configured value (1500) (03 §3.4, §15.7).

Inserts can set only a few fields · writes (ISCO-48)

/Customer/Post accepts 13 fields (Name, VatReg, Address, MobileNumber, PhoneNumber, Salesman, Street, City, Town, Referee, RefereeMobile, CustomerLevel, PriceClass; p. 9): no code, credit_limit, currency, discount, paymentTermsCode, contacts, classifications or coordinates, which the read model has (p. 7). /Item/Post accepts 15 fields with one unit (unit1) and one price (price1) (p. 20): no units 2-6, other price classes or purchase price (pp. 9-10). **Impact:** A 'new customer' tool cannot set a credit limit or currency, and a 'new item' tool cannot set a full price list; someone must finish the record in the desktop program, and Shamel's defaults for the missing fields are unknown.

Bawaba: Cannot be fixed by a gateway — needs ISCO. Bawaba omits unset fields so Shamel applies its own defaults, and shows the approver exactly what will be sent (03 §15.4-15.5).

Two different Voucher classes (and other name clashes) · documentation (ISCO-40, ISCO-41)

GetVoucher returns the full Voucher {header, items, cheqs, accounts} (p. 17), GetDelivery a Voucher without accounts (p. 16), and after GetExtraVouchers a different 'public class Voucher' with type, code, date, customer_code, amount, currency, rate, cr_dr, customer_balance_at_voucher_date (p. 18). GetVouchers says only 'return array of Voucher' (p. 17). There are also two different Result, AccountsheetFilters, Customer and Item classes (pp. 7, 9, 10, 12, 15, 19, 20). **Impact:** Tools cannot know whether GetVouchers returns full vouchers with lines or light summary rows until it is tested, which changes both its cost and which reports can be built from it.

Bawaba: Bawaba separates them as /v1/vouchers (full vouchers) and /v1/voucher-summaries (light rows) with distinct names; its decoder accepts the full variant and tolerates missing collections (03 §12.2-12.3).

What the API does not do, and why it matters

Item images are file paths, not image data · data access (ISCO-34)

GetItemsCatalog ('Get all items image') returns only item_code and image_path strings (p. 11). No endpoint returns the image bytes, and the format of image_path (web URL, local path or network share) is not stated. **Impact:** Catalog, sales-app or e-commerce tools cannot show product pictures from the API alone, and passing image_path on could reveal Shamel's server name or folder layout.

Bawaba: image_path is never exposed. The Connector fetches the image on the Shamel side, Bawaba re-encodes it (metadata stripped) and serves it through short-lived presigned /v1/media URLs; images stay disabled until the path format is confirmed (03 §10.5-10.6).

No version information or change notices · documentation (ISCO-50)

The manual carries no version number or date (pp. 1-20), paths have no version, and no endpoint reports the Shamel or API version. The name /Voucher/Post2 (p. 13) implies an older, undocumented /Voucher/Post. **Impact:** A Shamel update could rename fields or change behaviour without warning and silently break tools; nobody can tell whether the manual matches the installed version.

Bawaba: Tools get a stable /v1 contract with its own version, changelog and 6-month deprecation notice; a tolerant decoder and schema-drift metrics and alerts spot upstream changes; /Voucher/Post is never called (03 §3, §7). Knowing Shamel's version and change plans needs ISCO.

Ambiguous numbers: '0 = not requested' and amounts as text · data access (ISCO-14, ISCO-17, ISCO-31, ISCO-38)

Customer balances are filled only with getBalance=true (p. 7), and for vouchers 'If the header is not set or set to false, 0 is returned' (pp. 17-18), so 0 can mean 'not asked'. Account-statement credit, debit and balance are strings with no stated number format (pp. 13, 19); other amounts are double (floating point); Customer.price_class is a double although price classes have int ids (pp. 6-7). **Impact:** A tool may show a zero balance that is really unknown, or mis-parse amounts and get wrong totals.

Bawaba: Bawaba requests balances whenever it exposes them and never reports a 0 it did not ask for; all money, quantities and rates are exact decimal strings; statement strings are parsed with a strict grammar and the whole response fails safely on an unknown format (03 §4.6, §12.2).

Possible undocumented endpoints on the Shamel server · security (ISCO-07, ISCO-50, ISCO-55)

The manual documents 30 endpoints; /Voucher/Post2 (p. 13) implies an older /Voucher/Post, and nothing says whether other endpoints (older versions, update or delete, admin or help pages) exist or whether all of them require the token (p. 4). **Impact:** Anyone who can reach the Shamel API directly on the network might find and use endpoints nobody has reviewed, possibly ones that change data.

Bawaba: Bawaba and its Connector forward only a compiled allowlist of reviewed endpoints (never /Voucher/Post), so tools cannot reach anything else through Bawaba; with the Connector on the Shamel server, Shamel can be firewalled to accept only local calls if ISCO confirms that is safe (01 §8.1, 03 §7). Blocking or disabling such endpoints on Shamel itself needs ISCO.

No test environment for writes · writes (ISCO-51)

The manual describes no test company, demo database or sandbox mode (pp. 1-20); every insert goes to the live company data (pp. 8-9, 13-20). **Impact:** Write tools can only be tried for real on live accounting data, where mistakes cannot be deleted through the API.

Bawaba: Test tokens are routed to a built-in mock Shamel with realistic fake data, so tools can be built and tested safely; the owner switches a tool to a live token once trusted (00 §6, 03 §21). The mock is built from the manual, so each write type must still be verified once against real Shamel, ideally on a test copy from ISCO.

Inconsistent naming and typos · documentation (ISCO-13, ISCO-32, ISCO-35, ISCO-39)

Misspellings are part of the interface: sale_parice1-6 and orginal_code (p. 10), orgnization_code (pp. 6, 10, 17), GetBanksBranchs (p. 5), CustomerClassificaiton and notPayedCheqsBalance (p. 7), start_grantee_date (p. 12), refrenceId (p. 15). Styles are mixed (VATReg, creditType, salesman_code, nameEn); GetStores is printed as class City (p. 6); /CostCenter/Post says 'Insert new customer' (p. 18); four paths lack the leading slash (pp. 16-17); /Item/getItemDetails has a lower-case g (p. 11). Read and write models differ: orginal_code vs original_code, sale_parice1 vs price1 (pp. 10, 20), Customer read snake_case vs write PascalCase (pp. 7, 9). **Impact:** AI agents and developers easily use the 'corrected' spelling and get empty or ignored fields; mapping read results back into writes is error-prone.

Bawaba: /v1 uses one consistent snake_case naming that fixes the typos and documents every mapping; an optional legacy passthrough keeps Shamel's original names for existing integrations (00 §9, 03 §20).

GetLastPriceReport: ItemCode both mandatory and optional · documentation (ISCO-37)

The manual lists 'Fill Headers[ItemCode]', 'ItemCode is mandatory', 'Fill Headers[barcode]', 'ItemCode is optional' (p. 12). There is no customer filter, and each row's customer field is not explained (code or name). **Impact:** It is unclear whether a barcode alone works, and a 'last price to this customer' tool cannot ask Shamel for one customer; if customer is a name, rows cannot be matched to customer codes reliably.

Bawaba: Bawaba requires item_code and treats barcode as optional, adds customer, currency and date filters on the snapshot, and for customer-restricted tokens compares customer with allowed codes so a name never matches (fail closed) (03 §13.2).

13 HOW BAWABA FILLS THE GAPS

A gateway can fix a lot, but not everything

Bawaba sits in front of the Shamel API. It cannot add data Shamel does not expose, but it can make the existing API safe, fast and pleasant to build on.

Shamel limit	What Bawaba does about it
One all-powerful login; no permissions beyond salesman scoping	This is Bawaba's main purpose: tools never see Shamel credentials.
Salesman scoping can be switched off by the caller	getAll is not a client parameter on /v1; Bawaba sends it only when the token's policy allows it and the identity is not a pinned salesman user.
Unknown token lifetime and session behaviour; no logout	Bawaba holds one shared session per Shamel identity, logs in one at a time cluster-wide, uses a single session holder by default, re-logs in after an auth failure and...
No HTTPS documented; password and token may travel in plain text	The Bawaba Connector runs next to Shamel (ideally on the same server) and opens an outbound encrypted tunnel (WebSocket with inner mutual TLS 1.3), so Shamel needs no...
Every error is HTTP 500 with raw details	An outcome classifier sorts every upstream reply (auth failure via a calibrated signature, business rejection, server error, timeout); clients get proper status codes...
Write results come in three shapes with free-text errors	Bawaba normalises all three shapes, maps known Shamel messages to stable codes (duplicate_code, reference_not_found, validation_failed) and returns everything else as...
Parameters go in HTTP headers (proxy, size and Arabic-text problems)	Tools use a clean /v1 REST API with query parameters, path ids and JSON bodies.
Writes are JSON packed inside a form field	Tools send ordinary JSON bodies to /v1, checked against a strict schema (unknown fields rejected, decimals exact).
No pagination: lists return everything at once	Snapshot pagination: page 1 makes one live call, later pages reuse that result for about 60 s through sealed cursors, with totals, sorting and Arabic-aware search;...
No voucher-type filter and no voucher lookup by manual_code	/v1/vouchers adds type, manual_code, salesman, store and posted filters plus search, evaluated on the snapshot (same upstream cost as the unfiltered range).
Few server-side filters; no search or sort	Bawaba adds filters, sorting and Arabic-normalised search (alef/hamza forms, taa marbuta, diacritics) over the snapshot, plus combined endpoints such as customer 360 and...
lastupdatedate is date-only and exists on only three endpoints	/v1 offers updated_since on customers, items and stock, sending Shamel a safely earlier date (one day plus a 6-hour overlap) and documenting a 'superset' guarantee...
No change feed for vouchers, balances or deletions	Webhooks (milestone 3) detect changes by polling every 5 minutes: tiered voucher scans (last 3 days every cycle, 35-day hourly sweep via GetExtraVouchers, nightly 35-day...
Mistakes can only be fixed with offsetting documents	Approval queue: writes from selected tokens (the default for agent-requested tokens) wait for the owner's approval on the server; every write is audited with its full...
No idempotency: a retry can create a duplicate	Every /v1 write requires an Idempotency-Key; a repeat with the same key replays the stored result instead of calling Shamel again.
Undocumented or half-documented endpoints	These are exposed as provisional: a tolerant decoder accepts several plausible shapes (e.g.
Coded fields and enums without values	Bawaba publishes these as raw integer ids marked provisional and adds named values once known; journal writes stay unavailable until the debit_credit mapping is...
Inconsistent naming and typos	/v1 uses one consistent snake_case naming that fixes the typos and documents every mapping; an optional legacy passthrough keeps Shamel's original names for existing...
Two different Voucher classes (and other name clashes)	Bawaba separates them as /v1/vouchers (full vouchers) and /v1/voucher-summaries (light rows) with distinct names; its decoder accepts the full variant and tolerates...

A gateway can fix a lot, but not everything

GetLastPriceReport: ItemCode both mandatory and optional	Bawaba requires item_code and treats barcode as optional, adds customer, currency and date filters on the snapshot, and for customer-restricted tokens compares customer…
Unknown rate limits and impact on desktop users	A global cap on concurrent Shamel calls (default 8) with a fair queue per token, priority for interactive reads, at most 50% of slots for large calls and 25% for…
Item images are file paths, not image data	image_path is never exposed.
No version information or change notices	Tools get a stable /v1 contract with its own version, changelog and 6-month deprecation notice; a tolerant decoder and schema-drift metrics and alerts spot upstream…
Only three reports; little history	Partly: snapshot paging, voucher range slicing, voucher summaries and combined endpoints (customer 360, stock across stores) make computing from raw data practical.
Ambiguous numbers: '0 = not requested' and amounts as text	Bawaba requests balances whenever it exposes them and never reports a 0 it did not ask for; all money, quantities and rates are exact decimal strings; statement strings…
Possible undocumented endpoints on the Shamel server	Bawaba and its Connector forward only a compiled allowlist of reviewed endpoints (never /Voucher/Post), so tools cannot reach anything else through Bawaba; with the…
No test environment for writes	Test tokens are routed to a built-in mock Shamel with realistic fake data, so tools can be built and tested safely; the owner switches a tool to a live token once…

Needs ISCO, not a gateway

- **No update, cancel or delete operations**: Cannot be fixed by a gateway — needs ISCO.
- **Inserts can set only a few fields**: Cannot be fixed by a gateway — needs ISCO.
- **No lookup lists for many codes**: Cannot be fixed by a gateway — needs ISCO.

Where to read more

Bawaba design: `docs/design/` and the visual overview PDF. Open questions to ISCO: `08-isco-questions.md` (55 questions, Arabic + English).

A APPENDIX

Endpoint reference: all 30 endpoints

Every endpoint in ISCO's manual with its inputs, return type, page in the manual and the matching Bawaba route.

`/Main/Login` auth · POST (stated) · p. 4 Bawaba: not exposed (used only by Bawaba's session manager; /legacy/Main/Login is refused)	**Log a Shamel user in and get the session token that every other call must carry.** Inputs: password (form) · Returns: AuthResult (single object) "Keep the token to send it with every request" (p. 4); later calls send headers username + token, never the password.; Token lifetime, expiry, refresh, logout and the effect of a second login are undocumented (ISCO-01, ISCO-02).
`/Files/GetCurrencies` read · GET (assumed) · p. 4 Bawaba: GET /v1/currencies	**List all currencies defined in Shamel.** Inputs: none besides username/token · Returns: list of Currency "Get all currencies" (p. 4). No filter.; units and division are undocumented (ISCO-28).
`/Files/GetCurrencyRates` read · GET (assumed) · p. 5 Bawaba: GET /v1/currency-rates	**Get the latest 10 exchange rates.** Inputs: none besides username/token · Returns: list of CurrencyRate (latest 10) "Get latest 10 currency rate" (p. 5): fixed limit, no currency or date filter, so historical rates or the rate on a given date cannot be read.; Whether 10 means 10 in total or 10 per currency is not stated (ISCO-28).
`/Files/GetBanks` read · GET (assumed) · p. 5 Bawaba: GET /v1/banks	**List all banks.** Inputs: none besides username/token · Returns: list of Bank "Get All Banks" (p. 5). No filter.; Bank.code is what VoucherCheqs.bank_code refers to (p. 14, comment رقم البنك).
`/Files/GetBanksBranchs` read · GET (assumed) · p. 5 Bawaba: GET /v1/bank-branches	**List all bank branches.** Inputs: none besides username/token · Returns: list of BankBranch "Get all branchs" [sic] (p. 5). The path itself is misspelled (Branchs) and must be called as printed.; No filter by bank: all branches come back; bank_code links a branch to Bank.code (inferred).
`/Files/GetCities` read · GET (assumed) · p. 6 Bawaba: GET /v1/cities	**List all cities.** Inputs: none besides username/token · Returns: list of City "Get all cities" (p. 6).; Customer.city, Customer(write).City and VoucherHeader.city are int ids referring to City.id (inferred from types).
`/Files/GetStores` read · GET (assumed) · p. 6 Bawaba: GET /v1/stores	**List all stores (warehouses).** Inputs: none besides username/token · Returns: list of Store (the manual prints the class name as "City") "Get all stores" (p. 6), but the structure is printed as "class City" (copy-paste error).; type values are undocumented; car_code suggests some stores are vehicles for van sales (inferred); orgnization_code [sic] undocumented (ISCO-27, ISCO-49).
`/Files/GetPriceClasses` read · GET (assumed) · p. 6 Bawaba: GET /v1/price-classes	**Get the names of the 6 price classes.** Inputs: none besides username/token · Returns: list of PriceClass (6 rows) "Get the 6 price classes name" (p. 6).; Price class N is presumably Item.sale_pariceN and Customer.price_class = N (inferred; ISCO-29). Customer.price_class is typed double while PriceClass.id is int.
`/Files/GetSalesmen` read · GET (assumed) · p. 6 Bawaba: GET /v1/salesmen (provisional)	**Presumably lists salesmen; the manual prints only the heading.** Inputs: none besides username/token · Returns: undocumented Heading only (p. 6): no description, no inputs, no response model. Kind "read" is inferred from the name.; Salesman codes appear in AuthResult.salesmanCode (p. 4), Customer.salesman_code (p. 7), Customer(write).Salesman (p. 9) and VoucherHeader.salesman (p. 13).

Endpoint reference: all 30 endpoints

`/Customer/GetCustomers` read · GET (assumed) · p. 6 Bawaba: GET /v1/customers; GET /v1/customers/{code}	**List customers (by default only the logged-in salesman's), optionally only those changed since a date, with optional balances, classifications and contacts.** Inputs: lastupdatedate, code, vatreg, getBalance, getStopped, getClassifications, getContacts, getAll · Returns: list of Customer (read model: 31 scalar fields + classifications + contacts lists) · supports lastupdatedate Salesman scoping is implicit: "To get all customers even not the salesman's customers fill Header["getAll"]=true" (p. 7). Behaviour for a user with no salesman link is unknown (ISCO-25).; getStopped returns only stopped customers; whether the default (no header) includes stopped customers is not stated (ISCO-30).
`/Customer/GetCustomerBalances` read · GET (assumed) · p. 8 Bawaba: GET /v1/customer-balances; GET /v1/customers/{code}/balances	**Get the balances of one customer (by code or VAT number), or of all customers.** Inputs: customerCode, vatreg · Returns: list of CustomerBalance Header names printed without quotes (Header[customerCode], Header[vatreg]) (p. 8).; One row per customer per currency is inferred from the currency field; pos, date and rate are undocumented (ISCO-31).
`/Customer/Post` write · POST (stated) · p. 8 Bawaba: POST /v1/customers (M2)	**Insert a new customer; Shamel assigns and returns the new customer code.** Inputs: obj (json-in-form) · Returns: PostResponse (JSON) "Insert new customer" (p. 8). "If the process succeeded, the value will contain the inserted code of the customer" (p. 9): the code is assigned by Shamel; there is no code field in the write model.; Insert only: no update or delete endpoint exists for customers.
`/Customer/GetCustomerPricesAsRows` read · GET (assumed) · p. 16 Bawaba: GET /v1/customer-prices; GET /v1/customers/{code}/prices	**Presumably returns customer- or price-class-specific item prices and discounts as rows; the manual states no purpose.** Inputs: none besides username/token · Returns: unnamed price-row object, presumably a list (manual: "Return the following object") Path printed without a leading slash (p. 16); assumed to be /Customer/GetCustomerPricesAsRows (ISCO-13).; No inputs documented: presumably every row for every customer in one call (ISCO-21, ISCO-33).
`/Item/GetItems` read · GET (assumed) · p. 9 Bawaba: GET /v1/items; GET /v1/items/{code}	**List items (all, or only those changed since a date), optionally filtered by item code, serial, store or model.** Inputs: lastupdatedate, serial, itemCode, store, model · Returns: list of Item (read model, 51 fields) · supports lastupdatedate Field names kept as printed, including sale_parice1..6, orginal_code and orgnization_code (pp. 9-10).; Sensitive fields come back to every caller: purchase_price, credit_account, debit_account (p. 10).
`/Item/GetItemsStores` read · GET (assumed) · p. 10 Bawaba: GET /v1/stock; GET /v1/items/{code}/stock	**Get stock quantities per item per store, for all items or one item.** Inputs: lastupdatedate, itemcode · Returns: list of ItemStore · supports lastupdatedate "Get all items quantities" (p. 10).; No store filter: all stores come back (ISCO-36).
`/Item/getItemDetails` read · GET (assumed) · p. 11 Bawaba: GET /v1/items/{code}?include=stock; GET /v1/items/{code}/overview	**Get one item together with its quantities in each store.** Inputs: itemCode · Returns: ItemDetails (manual says "It return list of this structure", but the shape is one item + its stock rows) The action name is printed with a lower-case g (p. 11); routing is presumably case-insensitive (ISCO-13).; Decode both a single object and an array, since the manual says "list" for a single-object shape.
`/Item/GetItemsGroups` read · GET (assumed) · p. 11 Bawaba: GET /v1/item-groups; GET /v1/item-groups/tree	**List item groups (a tree via parent codes).** Inputs: none besides username/token · Returns: list of ItemGroup Comment on code: "Item has itemGroupCode field that refer to this field" (p. 11), but the Item field is actually item_group_code; the parent link is parentCode here and item_group_father_code in Item (ISCO-35).; level is presumably the depth in the tree (inferred). Not listed in the p. 2 overview.
`/Item/GetItemsCatalog` read · GET (assumed) · p. 11 Bawaba: GET /v1/item-images; GET /v1/items/{code}/images (bytes via GET /v1/media/{media_id})	**List the image path of every item (the item catalogue).** Inputs: none besides username/token · Returns: list of ItemCatalog "Get all items image" (p. 11). No item filter.; image_path format is undocumented (URL, local path or UNC path) and may reveal the Shamel server; no endpoint returns image bytes (ISCO-34).

Endpoint reference: all 30 endpoints

`/Item/GetItemsSerials` read · GET (assumed) · p. 11 Bawaba: GET /v1/item-serials; GET /v1/items/{code}/serials	**List serial-numbered item units, optionally for one customer or one serial number.** Inputs: customer, serial · Returns: list of ItemSerial "Get all items with serials only" (p. 11).; No item-code filter. status values are undocumented (ISCO-49). start_grantee_date / end_grantee_date are misspellings of guarantee (warranty dates).
`/Item/Post` write · POST (stated) · p. 19 Bawaba: POST /v1/items (M2)	**Insert a new item with a caller-chosen code, one main unit and one price.** Inputs: obj (json-in-form) · Returns: PostResponse (JSON) "Insert new Item" (p. 19). Insert only: no update or delete endpoint for items.; Unlike Customer/Post, the manual does not say what value contains (inferred: the item code) (ISCO-48).
`/Report/GetLastPriceReport` report · GET (assumed) · p. 12 Bawaba: GET /v1/reports/last-prices	**Last-prices report for an item: recent transaction dates, prices, quantities and customers.** Inputs: ItemCode, barcode · Returns: list of LastPriceRepColumn Self-contradiction on p. 12: "ItemCode is mandatory" then "ItemCode is optional" after the barcode line (ISCO-37).; No customer filter: unclear whether it is global or per customer, and whether customer holds a code or a name (ISCO-37).
`/Report/GetAccountsheet` report · POST (stated) · p. 12 Bawaba: GET /v1/reports/account-statement	**Account statement (debits, credits, running balance) for a customer over a date range.** Inputs: obj (json-in-form) · Returns: list of AccountsheetColumns A read carried in a form POST body (p. 12).; Requiredness of the filter fields is not stated; the manual gives no field meanings (all inferred from names).
`/Voucher/Post2` write · POST (stated) · p. 13 Bawaba: POST /v1/vouchers/{voucher_type} (M2)	**Insert a new voucher of any of the 13 documented types (order, invoice, return, receipt, payment, notes, journal, deliveries, stock-out).** Inputs: obj (json-in-form) · Returns: Result (voucher variant, JSON) "Insert new voucher" (p. 13). "If the process succeeded, the refrenceId will contain the inserted code of the voucher" (p. 15). ErrorMessage is a string array here, unlike PostResponse.; Insert only: no update, delete, cancel or void endpoint; corrections presumably need a compensating voucher (e.g. credit note, return) or a manual fix in Shamel (inferred).
`/Voucher/GetDelivery` read · GET (assumed) · p. 16 Bawaba: GET /v1/deliveries/prepared	**Get the delivery that has been prepared for the logged-in salesman.** Inputs: none besides username/token · Returns: Voucher (delivery variant: header, items, cheqs; single object) "Get the delivery that is prepared to the salesman"; "It return object of Voucher class" (p. 16). No accounts list.; Behaviour with zero or several prepared deliveries, the voucher type returned, and whether the call changes state are unknown (ISCO-41).
`/Voucher/GetStatus` read · GET (assumed) · p. 16 Bawaba: GET /v1/order-status	**Get the status of an order: not ready, ready, closed, sent partially or sent.** Inputs: manual · Returns: order status 0..4 (0 not ready, 1 ready, 2 closed, 3 sent partially, 4 sent); representation undocumented "Return the order status" (p. 16); English labels only.; Response shape (bare number or JSON), the result for an unknown value, duplicate manual codes and non-order types are undocumented (ISCO-42).
`/Voucher/GetVoucher` read · GET (assumed) · p. 17 Bawaba: GET /v1/vouchers/{code}	**Fetch one voucher (header, lines, cheques, accounts) by its code, optionally with the customer's balance at that time.** Inputs: voucher_code, getCustomerBalance · Returns: Voucher (full: header, items, cheqs, accounts) Path printed without a leading slash (p. 17) (ISCO-13).; The header name contains an underscore, which nginx and some proxies drop by default (ISCO-11).
`/Voucher/GetVouchers` read · GET (assumed) · p. 17 Bawaba: GET /v1/vouchers	**List vouchers in a date range, optionally for one customer and/or currency.** Inputs: filters, getCustomerBalance · Returns: array of Voucher (which Voucher class is not stated; presumably the full one) Path printed without a leading slash (p. 17) (ISCO-13).; JSON inside a request header: header size limits and non-ASCII values are fragile (ISCO-11).

Endpoint reference: all 30 endpoints

`/Voucher/GetExtraVouchers` read · GET (assumed) · p. 17 Bawaba: GET /v1/voucher-summaries	**Presumably a lightweight list of vouchers in a date range with amount, direction and the customer's balance at each voucher date; the manual does not explain it.** Inputs: filters, getCustomerBalance · Returns: array of short Voucher (printed "public class Voucher"; a different class from the full one) Path printed without a leading slash (p. 17) (ISCO-13).; Which voucher types are included and what "extra" means are unknown; cr_dr values undocumented (ISCO-40, ISCO-49).
`/CostCenter/Post` write · POST (stated) · p. 18 Bawaba: POST /v1/cost-centers (M2)	**Insert a new cost center with a caller-chosen code.** Inputs: obj (json-in-form) · Returns: Result (cost-center variant, JSON): no identifier returned Text says "Insert new customer" and "put the customer info" (p. 18): copy-paste errors; the body is CostCenterDTO.; Returns only Success and ErrorMessage (p. 19); the created code can only be the code sent. Duplicate-code behaviour unknown (ISCO-39, ISCO-43).
`/CostCenter/GetCostCenterAccountsheet` report · POST (stated) · p. 19 Bawaba: GET /v1/reports/cost-center-statement	**Account statement for a range of cost centers (optionally one account) over a date range.** Inputs: filters (json-in-form) · Returns: list of AccountsheetColumns (same columns as /Report/GetAccountsheet) Lives under the CostCenter controller; domain set to reports to match the manual's own report list (p. 3).; type filter values and what account accepts are undocumented (ISCO-39, ISCO-49). Amounts are strings (same as GetAccountsheet, ISCO-38).

B APPENDIX

Data dictionary: every record and field

All record types in ISCO's manual with their fields exactly as printed (including original spellings), what each means, and where that meaning comes from.

MANUAL stated in the manual MANUAL-AR from an Arabic comment INFERRED our reading UNDOCUMENTED no meaning given

Login result نتيجة تسجيل الدخول

`AuthResult` `/Main/Login · p. 4`

What Shamel sends back when a user logs in with username and password (form POST, p. 4). It carries the session token that every other request must send, plus who the user is and which salesman they are linked to.

Field	Type	Meaning	Source	Note
`success`	`bool`	Whether the login succeeded	INFERRED	From the name. How a failed login is reported (success=false with HTTP 200, or HTTP 500) is not stated (ISCO-03).
`token`	`string`	Session token to send with every later request	MANUAL	"Keep the token to send it with every request" (p. 4); sent as header `token` together with header `username` (p. 4). "Token: you could have it by call login method" (p. 2). Lifetime/expiry undocumented (ISCO-01).
`userId`	`int`	Shamel user number	INFERRED	From the name.
`userCode`	`string`	Shamel user code	INFERRED	From the name. Whether the `username` header must carry the login name, userCode or userId is unconfirmed (ISCO-03).
`userName`	`string`	The user's name	INFERRED	From the name; display name vs login name not stated.
`salesmanCode`	`string`	Salesman linked to this user	INFERRED	salesmanCode → Salesman. Explains why GetCustomers returns only "the salesman's customers" unless getAll=true (p. 7). Scoping on other endpoints unknown (ISCO-25).
`userIsDisabled`	`bool`	The user account is disabled	INFERRED	From the name. Whether a disabled user can still get a usable token is unknown (ISCO-03).

Data dictionary: every record and field

Currency عملة `Currency` /Files/GetCurrencies · p. 4

A currency defined in Shamel ("Get all currencies", pp. 4–5). It is the lookup list that amounts, prices and balances point to through the currency code.

Field	Type	Meaning	Source	Note
`id`	`int`	Numeric currency id	INFERRED	From the name. Other records refer to currencies by the text code, not by this id (ISCO-28).
`code`	`string`	Currency code	INFERRED	Join key for CurrencyRate.currency_code, Customer.currency, CustomerBalance.currency and price rows. Vouchers use codes like "NIS", "$" (p. 13), so not necessarily ISO codes (ISCO-28 b).
`currency_name`	`string`	Currency name	INFERRED	Printed "stringcurrency_name" (missing space, p. 5).
`units`	`string`	Not explained; possibly the main unit name (e.g. shekel)	UNDOCUMENTED	ISCO-28 (a).
`division`	`string`	Not explained; possibly the sub-unit name (e.g. agora)	UNDOCUMENTED	ISCO-28 (a).

Exchange rate سعر صرف `CurrencyRate` /Files/GetCurrencyRates · p. 5

An exchange-rate entry. The endpoint returns only the "latest 10 currency rate" (p. 5) with no date or currency filter, so it gives current rates, not history. Whether that is 10 in total or 10 per currency is not stated (ISCO-28 c).

Field	Type	Meaning	Source	Note
`currency_code`	`String`	Currency this rate is for	INFERRED	currency_code → Currency.code. Type printed "String" (capital S).
`rate`	`double`	Exchange rate	INFERRED	From the name. Direction (base units per 1 unit of this currency?) not stated.
`date`	`string`	Date of the rate	INFERRED	yyyy-MM-dd per the global date rule (p. 4).
`is_base`	`bool`	Marks the base (local) currency	INFERRED	From the name. Exact meaning asked in ISCO-28 (d).

Bank بنك `Bank` /Files/GetBanks · p. 5

A bank defined in Shamel ("Get All Banks", p. 5; Arabic capability "سحب البنوك وفروعها", pull banks and their branches, p. 2). It identifies the bank of cheques.

Field	Type	Meaning	Source	Note
`code`	`string`	Bank code	INFERRED	Likely what VoucherCheqs.bank_code ("رقم البنك", bank number, p. 14) refers to.
`name`	`string`	Bank name	INFERRED	From the name.

Data dictionary: every record and field

Bank branch فرع بنك `BankBranch` `/Files/GetBanksBranchs · p. 5`

A branch of a bank ("Get all branchs" [sic], p. 5). The path is misspelled /Files/GetBanksBranchs; it returns all branches of all banks and cannot be filtered by bank.

Field	Type	Meaning	Source	Note
`code`	`string`	Branch code	INFERRED	Likely what VoucherCheqs.branch_code ("رقم فرع البنك", bank branch number, p. 14) refers to.
`name`	`string`	Branch name	INFERRED	From the name.
`bank_code`	`string`	Bank this branch belongs to	INFERRED	bank_code → Bank.code. Printed "stringbank_code" (missing space, p. 5).

City مدينة `City` `/Files/GetCities · p. 6`

A city in Shamel's city list ("Get all cities", p. 6). Customers and vouchers point to it by number.

Field	Type	Meaning	Source	Note
`id`	`int`	City number	INFERRED	Customer.city, Customer(write).City ("رقم المدينة", city number, p. 9) and VoucherHeader.city (p. 13) refer to it.
`name`	`string`	City name	INFERRED	Also copied into Customer.city_name.

Store (warehouse) مستودع `Store` `/Files/GetStores · p. 6`

A store (warehouse) where stock is held ("Get all stores", p. 6; Arabic capability "سحب المستودعات", pull warehouses, p. 2). The manual prints this class as `class City`, a copy-paste error; "Store" is our name for it.

Field	Type	Meaning	Source	Note
`id`	`int`	Store number	INFERRED	Referenced by ItemStore.store, VoucherItems.store, VoucherHeader.order_store and the `store` filter header of GetItems.
`name`	`string`	Store name	INFERRED	From the name.
`type`	`int`	Store type	INFERRED	Values undocumented (ISCO-27 a, ISCO-49).
`car_code`	`string`	Vehicle code, probably for van (mobile sales) stores	INFERRED	From the name only (ISCO-27 b).
`orgnization_code`	`string`	Not explained; probably an organisation or branch code	UNDOCUMENTED	[sic] "orgnization". ISCO-27 (c).

Price class فئة سعر `PriceClass` `/Files/GetPriceClasses · p. 6`

One of Shamel's six named price levels ("Get the 6 price classes name", p. 6; Arabic capability "سحب فئات الاسعار", p. 2). Each customer is assigned one.

Field	Type	Meaning	Source	Note
`id`	`int`	Price class number (1–6)	INFERRED	Presumably matches Item.sale_parice1..6 and Customer.price_class (ISCO-29 a).
`name`	`string`	Price class name	MANUAL	"Get the 6 price classes name" (p. 6).

Data dictionary: every record and field

Salesman مندوب `Salesman` /Files/GetSalesmen · p. 6

A sales representative (مندوب). UNDOCUMENTED: the manual prints only the heading /Files/GetSalesmen (p. 6) with no description, no inputs and no fields, and the endpoint is missing from the Arabic capability list (p. 2). Its response must come from ISCO or a live capture (ISCO-26, priority).

No fields documented.

Customer (read) زبون `Customer` /Customer/GetCustomers · p. 7

A customer account as returned by /Customer/GetCustomers (pp. 6–7): identity, phone and address, salesman, price class and credit terms, tax status, GPS, and optionally balances, classifications and contacts. By default only the logged-in user's salesman's customers come back; getAll=true returns all (p. 7). Filters: lastupdatedate (changed since), code, vatreg, getStopped.

Field	Type	Meaning	Source	Note
`code`	`string`	Customer code (unique key)	INFERRED	Used as the `code` filter: "to get one customer with the filled 'code'" (p. 7). Assigned by Shamel on insert (PostResponse.value, p. 9). Same key as customerCode / customer_code elsewhere.
`name`	`string`	Customer name	INFERRED	Write field Name = "اسم الزبون" (p. 9).
`phone`	`string`	Phone number	INFERRED	Probably from write field PhoneNumber ("تلفون 1") (ISCO-32 b).
`street`	`string`	Street address	INFERRED	Probably from write field Address ("الشارع") (ISCO-32 b).
`street2`	`string`	Second street / address line	INFERRED	Probably from write field Street ("شارع 2") (ISCO-32 b).
`referee1`	`string`	Referee 1 (person who introduces or vouches for customer)	INFERRED	Write field Referee = "معرف 1" (p. 9) (ISCO-32 a).
`referee2`	`string`	Second referee, or referee 1's phone (unclear)	INFERRED	May hold write field RefereeMobile ("تلفون معرف 1") (ISCO-32 b).
`salesman_code`	`string`	Salesman responsible for this customer	INFERRED	salesman_code → Salesman. Drives default scoping: without getAll=true only "the salesman's customers" return (p. 7).
`price_class`	`double`	Price class assigned to the customer	INFERRED	price_class → PriceClass.id. Typed double here but int in the write model (ISCO-29 c).
`credit_limit`	`double`	Credit limit	INFERRED	Sensitive. Cannot be set through /Customer/Post (ISCO-48 a).
`discount`	`double`	Default customer discount	INFERRED	Percentage or amount not stated (ISCO-32 a).
`city`	`int`	City number	INFERRED	city → City.id.
`currency`	`string`	Account currency code	INFERRED	currency → Currency.code.
`level`	`int`	Customer level	INFERRED	Write field CustomerLevel = "مستوى الزبون" (p. 9). Values undocumented (ISCO-49).
`town`	`string`	Town / village code	INFERRED	Write field Town = "كود القرية" (village code, p. 9). No endpoint lists towns.
`city_name`	`string`	City name (copy of City.name)	INFERRED	From the name.

Data dictionary: every record and field

Customer (read) (continued)

Field	Type	Meaning	Source	Note
citizen_code	string	Not explained	UNDOCUMENTED	ISCO-32 (a).
attr	int	Not explained	UNDOCUMENTED	Enum values unknown (ISCO-32 a, ISCO-49).
man_name	string	Not explained; possibly contact or responsible person's name	UNDOCUMENTED	ISCO-32 (a).
trans_permissions	int	Not explained; possibly allowed-transaction flags	UNDOCUMENTED	ISCO-32 (a), ISCO-49.
VATReg	string	VAT registration (licensed-dealer) number	INFERRED	Filtered by the `vatreg` header: "to filter on vatreg field" (p. 7). Write field VatReg = "مشتغل مرخص الزبون" (p. 9).
creditType	int	Not explained; possibly credit or payment type	UNDOCUMENTED	ISCO-32 (a), ISCO-49.
balance	double	Customer's current balance	MANUAL	Filled only with getBalance="true" (p. 7); otherwise presumably 0, indistinguishable from a real zero (ISCO-31 a). Sensitive.
notPayedCheqsBalance	double	Total of the customer's cheques not yet paid	MANUAL	[sic] "customer's not payed cheqs balance" (p. 7). Only with getBalance="true".
returnedCheqsBalance	double	Total of the customer's returned (bounced) cheques	MANUAL	"customer's returned cheqs balance" (p. 7). Only with getBalance="true".
stopped	bool	Customer is stopped (blocked / inactive)	INFERRED	getStopped="true" returns "only stopped customers" (p. 7). Whether the default call includes stopped customers is unknown (ISCO-30).
latitude	string	GPS latitude (as text)	INFERRED	From the name.
longitude	string	GPS longitude (as text)	INFERRED	From the name.
id_number	string	Identity number	INFERRED	Personal ID vs company registration unclear (ISCO-32 a).
taxable	bool	Customer is subject to tax	INFERRED	From the name.
paymentTermsCode	string	Payment terms code	INFERRED	From the name. No endpoint lists payment terms.
classifications	List<CustomerClassificaiton>	Customer's classifications	MANUAL	Filled only with getClassifications="true": "to return customer classifications" (p. 7). → CustomerClassificaiton [sic].

Customer (read) (continued)

Field	Type	Meaning	Source	Note
contacts	List<CustomerContact>	Customer's contacts	MANUAL	Filled only with getContacts="true": "to return customer's contacts" (p. 7). → CustomerContact.

Data dictionary: every record and field

Customer classification تصنيف الزبون `CustomerClassificaiton` /Customer/GetCustomers · p. 7

A classification tag attached to a customer (e.g. a segment or group). Returned inside Customer.classifications only when getClassifications="true" (p. 7). The class name is misspelled in the manual ("Classificaiton").

Field	Type	Meaning	Source	Note
pos	int	Position / slot number of the classification	INFERRED	From the name.
name	string	Classification name	INFERRED	Printed on p. 8.

Customer contact جهة اتصال الزبون `CustomerContact` /Customer/GetCustomers · p. 8

A contact entry for a customer (a person, phone, email or similar). Returned inside Customer.contacts only when getContacts="true" (p. 7).

Field	Type	Meaning	Source	Note
pos	int	Position / order of the contact	INFERRED	From the name.
name	string	Contact name or label	INFERRED	From the name.
detail	string	Contact value (e.g. number, email, address)	INFERRED	From the name.
type	int	Contact type number	INFERRED	Values undocumented (ISCO-49).
typeName	string	Contact type name (label for type)	INFERRED	From the name.

Customer balance رصيد الزبون `CustomerBalance` /Customer/GetCustomerBalances · p. 8

A customer balance row ("Get list of customer balances", p. 8; Arabic capability "سحب ارصدة الزبائن", p. 2). Apparently one row per customer per currency. Empty customerCode returns all customers' balances; vatreg finds a customer by VAT number.

Field	Type	Meaning	Source	Note
code	string	Customer code	INFERRED	code → Customer.code.
pos	int	Not explained; possibly row or currency order	UNDOCUMENTED	ISCO-31 (b).
currency	string	Currency of this balance	INFERRED	currency → Currency.code.
date	string	Not explained; possibly balance or last-movement date	UNDOCUMENTED	ISCO-31 (b).
balance	double	Customer balance amount	MANUAL	Endpoint returns "customer balances" (p. 8). Sign (owed vs credit) and whether cheques/unposted vouchers are included not stated (ISCO-31 c).
rate	double	Exchange rate, presumably of the row's currency	INFERRED	ISCO-31 (b).

Data dictionary: every record and field

Customer (new-customer form) زبون جديد `Customer` → /Customer/Post · p. 9

The data sent to create a customer via /Customer/Post ("Insert new customer", p. 8), as a JSON string in form field `obj`. Same class name as the read model but different (PascalCase) fields; there is no customer code because Shamel assigns it and returns it in PostResponse.value (p. 9). All meanings come from Arabic comments.

Field	Type	Meaning	Source	Note
`Name`	`string`	Customer name	MANUAL-AR	Arabic: «اسم الزبون» (p. 9). Read field: name.
`VatReg`	`string`	Customer's licensed-dealer (VAT registration) number	MANUAL-AR	Arabic: «مشتغل مرخص الزبون». Printed "stringVatReg" (missing space). Read field: VATReg.
`Address`	`string`	Street	MANUAL-AR	Arabic: «الشارع», printed without "//". Probably read field street (ISCO-32 b).
`MobileNumber`	`string`	Fax (Arabic comment contradicts the field name)	MANUAL-AR	Arabic: «فاكس». Mobile vs fax unclear; no read counterpart (ISCO-32 c).
`PhoneNumber`	`string`	Phone 1	MANUAL-AR	Arabic: «تلفون 1». Probably read field phone.
`Salesman`	`string`	Default salesman; leave empty if no salesmen exist	MANUAL-AR	Arabic: «المندوب الافتراضي – في حال عدم وجود مندوبين لا يتم تعبئته». Salesman → Salesman; read field salesman_code.
`Street`	`string`	Street 2	MANUAL-AR	Arabic: «شارع 2». Probably read field street2 (careful: Street is not street).
`City`	`int`	City number	MANUAL-AR	Arabic: «رقم المدينة». City → City.id.
`Town`	`string`	Village code	MANUAL-AR	Arabic: «كود القرية». Read field town; no endpoint lists towns.
`Referee`	`string`	Referee 1	MANUAL-AR	Arabic: «معرف 1». Read field referee1.
`RefereeMobile`	`string`	Referee 1's phone	MANUAL-AR	Arabic: «تلفون معرف 1». Possibly read field referee2 (ISCO-32 b).
`CustomerLevel`	`int`	Customer level	MANUAL-AR	Arabic: «مستوى الزبون». Read field level; values undocumented (ISCO-49).
`PriceClass`	`int`	Price class number	MANUAL-AR	Arabic: «رقم فئة السعر». PriceClass → PriceClass.id; int here, double in the read model.

Insert result نتيجة الإدخال `PostResponse` /Customer/Post, /Item/Post · p. 9

The JSON result of a customer or item insert ("It return the following class as Json", pp. 9 and 20): success flag, error text and the new record's code.

Field	Type	Meaning	Source	Note
`Success`	`bool`	Whether the insert succeeded	INFERRED	From the name. Failures may also arrive as HTTP 500 with error details (p. 4) (ISCO-19).
`ErrorMessage`	`string`	Error text when the insert failed	INFERRED	A single string here; the voucher Result uses string[] (p. 15).
`value`	`string`	Code of the newly inserted customer	MANUAL	"If the process succeeded, the value will contain the inserted code of the customer" (p. 9). For /Item/Post not stated (ISCO-48 b).

Data dictionary: every record and field

Customer price row سعر وخصم خاص بالزبون `CustomerPriceRow` Customer/GetCustomerPricesAsRows · p. 16

A special price/discount row from Customer/GetCustomerPricesAsRows (pp. 16–17, path printed without leading slash). The manual gives no class name ("CustomerPriceRow" is our name), no purpose and no inputs, only "Return the following object". The fields suggest item prices and discounts per customer or price class. Not in the Arabic capability list (p. 2).

Field	Type	Meaning	Source	Note
`code`	`string`	Not explained; possibly row or price-list code	UNDOCUMENTED	ISCO-33 (c).
`price_class_name`	`string`	Price class name	INFERRED	price_class_name → PriceClass.name. Rows with empty customer_code may be class prices (ISCO-33 e).
`customer_code`	`string`	Customer the price applies to	INFERRED	customer_code → Customer.code.
`stock_code`	`string`	Item code	INFERRED	stock_code → Item.code; unconfirmed (ISCO-33 d).
`discount`	`double`	Discount	INFERRED	Percent or amount presumably set by discount_type.
`discount_type`	`int`	Discount type (e.g. percent or amount)	INFERRED	Values undocumented (ISCO-33 c, ISCO-49).
`price`	`double`	Special price	INFERRED	From the name.
`currency_code`	`string`	Currency of the price	INFERRED	currency_code → Currency.code.
`main_unit`	`string`	Item's main unit name	INFERRED	From the name; printed on p. 17.
`orgnization_code`	`string`	Not explained; probably an organisation or branch code	UNDOCUMENTED	[sic] "orgnization".
`tax`	`double`	Tax (rate or amount not stated)	INFERRED	ISCO-33 (c).

Data dictionary: every record and field

Item / product (read) صنف

`Item` `/Item/GetItems, /Item/getItemDetails · p. 9`

The full item (product) card as Shamel holds it: names, up to 6 units with conversion factors, 6 sale prices with 6 currency fields, 6 per-unit prices, barcode and other codes, group, status flags, purchase price and two GL accounts (pp. 9-10, 51 fields). It is returned as a list by /Item/GetItems and embedded as ItemDetails.itm by /Item/getItemDetails; every stock, catalog, serial and sales record joins to it by code.

Field	Type	Meaning	Source	Note
code	string	Item code (item number)	MANUAL-AR	Arabic comment on the same-named write-model field: 'رقم الصنف' (item number), p. 20. Filtered by GetItems header itemCode (p. 9). Referenced by ItemStore.item_code, ItemCatalog.item_code, ItemSerial.itemCode, VoucherItems.code.
name	string	Item name (usually Arabic)	MANUAL-AR	Write-model comment 'اسم الصنف' (item name), p. 20. Arabic text is typical (Q48).
nameEn	string	English item name	INFERRED	From the field name. camelCase, unlike most Item fields (Q45). The write model has no English-name field.
unit1	string	Main (base) unit name	MANUAL-AR	Write-model comment 'وحدة الصنف الرئيسية' (item's main unit), p. 20. unit2..unit6 convert to it via unitN_relation. ItemStore.main_unit presumably equals it.
unit2	string	Second (alternative) unit name	INFERRED	Pairs with unit2_relation and unit_price_2. Empty when unused (assumed). VoucherItems.unit is an int, presumably an index 1..6 into unit1..unit6 (ISCO-49).
unit3	string	Third (alternative) unit name	INFERRED	Pairs with unit3_relation and unit_price_3.
unit4	string	Fourth (alternative) unit name	INFERRED	Pairs with unit4_relation and unit_price_4.
unit5	string	Fifth (alternative) unit name	INFERRED	Pairs with unit5_relation and unit_price_5.
unit6	string	Sixth (alternative) unit name	INFERRED	Pairs with unit6_relation and unit_price_6.
unit2_relation	double	Conversion factor between unit2 and the main unit	INFERRED	Direction undocumented (assumed: how many unit1 in one unit2, e.g. 12 pieces per box). There is no unit1_relation because unit1 is the base.
unit3_relation	double	Conversion factor between unit3 and the main unit	INFERRED	Same reading and caveat as unit2_relation.
unit4_relation	double	Conversion factor between unit4 and the main unit	INFERRED	Same reading and caveat as unit2_relation.
unit5_relation	double	Conversion factor between unit5 and the main unit	INFERRED	Same reading and caveat as unit2_relation.

Data dictionary: every record and field

Item / product (read) (continued)

Field	Type	Meaning	Source	Note
unit6_relation	double	Conversion factor between unit6 and the main unit	INFERRED	Same reading and caveat as unit2_relation.
sale_parice1	double	Sale price for price class 1	INFERRED	[sic 'price'], name kept. Six prices match the 6 price classes ('Get the 6 price classes name', /Files/GetPriceClasses, p. 6); a customer's class is Customer.price_class. Mapping unconfirmed (ISCO-29a). Currency possibly curr1 (ISCO-29b).
sale_parice2	double	Sale price for price class 2	INFERRED	[sic]. See sale_parice1. Currency possibly curr2.
sale_parice3	double	Sale price for price class 3	INFERRED	[sic]. See sale_parice1. Currency possibly curr3.
sale_parice4	double	Sale price for price class 4	INFERRED	[sic]. See sale_parice1. Currency possibly curr4.
sale_parice5	double	Sale price for price class 5	INFERRED	[sic]. See sale_parice1. Currency possibly curr5.
sale_parice6	double	Sale price for price class 6	INFERRED	[sic]. See sale_parice1. Currency possibly curr6.
barcode	string	Barcode	MANUAL-AR	Write-model comment 'الباركود' (barcode), p. 20. /Report/GetLastPriceReport accepts a barcode header (p. 12). Only one barcode per item is exposed.
factory_code	string	Manufacturer's (factory) number	MANUAL-AR	Write-model comment 'رقم المصنع' (factory number), p. 20.
orginal_code	string	Original (part) number	INFERRED	[sic 'original'], name kept. The write model spells it original_code, commented 'الرقم الاصلي' (original number), p. 20 (Q29).
stopped	int	Item stopped (discontinued) flag, 0/1 assumed	INFERRED	An int here but bool on Customer (Q44). Values undocumented (ISCO-49). Whether GetItems includes stopped items is not stated.
paused	int	Item paused (temporarily frozen) flag, 0/1 assumed	INFERRED	How it differs from stopped is undocumented (ISCO-49).

Data dictionary: every record and field

Item / product (read) (continued)

Field	Type	Meaning	Source	Note
type	int	Item type	MANUAL-AR	Write-model comment 'نوع الصنف' (item type), p. 20. Values undocumented and no lookup endpoint (Q42, ISCO-48b, ISCO-49).
has_serial	int	Item tracked by serial number, 0/1 assumed	INFERRED	Such items presumably appear in /Item/GetItemsSerials (ItemSerial). Values unconfirmed (ISCO-49).
has_measure	int	Item has sizes (measures), 0/1 assumed	INFERRED	VoucherItems.measure is commented 'not used' (غير مستخدم), p. 14, so sizes cannot be sent on vouchers.
has_color	int	Item has colour variants, 0/1 assumed	INFERRED	VoucherItems.color is commented 'not used' (غير مستخدم), p. 14.
has_expiry	int	Item tracks expiry dates, 0/1 assumed	INFERRED	No endpoint returns expiry or batch data.
eval_unit	int	Unknown; name suggests the unit used for stock valuation	UNDOCUMENTED	Possibly an index 1..6 into unit1..unit6 (guess). Values undocumented (ISCO-49).
purchase_price	double	Purchase (cost) price	INFERRED	Last purchase price vs average cost is unknown (ISCO-29d). Sensitive: margin data, hide from sales-facing tools.
item_group_code	string	Code of the item's group	INFERRED	→ ItemGroup.code. The ItemGroup comment calls it 'itemGroupCode' (p. 11; Q28, ISCO-35c). The write model calls it group.
item_group_name	string	Name of the item's group	INFERRED	Denormalised copy of ItemGroup.name (assumed).
item_group_father_code	string	Code of the parent group	INFERRED	→ ItemGroup.code of the parent; corresponds to ItemGroup.parentCode (unconfirmed, ISCO-35c). Only one parent level is carried on the item.
item_group_father_name	string	Name of the parent group	INFERRED	Denormalised copy of the parent ItemGroup.name (assumed).
unit_price_1	double	Price of unit 1 (main unit)	INFERRED	Undocumented. How it differs from sale_parice1 is unknown (ISCO-29b). The write-model price1 'سعر الوحدة الرئيسية' (main-unit price) may map here or to sale_parice1.

Data dictionary: every record and field

Item / product (read) (continued)

Field	Type	Meaning	Source	Note
unit_price_2	double	Price of unit 2	INFERRED	Pairs with unit2 (per unit, not per class, assumed). See unit_price_1.
unit_price_3	double	Price of unit 3	INFERRED	Pairs with unit3. See unit_price_1.
unit_price_4	double	Price of unit 4	INFERRED	Pairs with unit4. See unit_price_1.
unit_price_5	double	Price of unit 5	INFERRED	Pairs with unit5. See unit_price_1.
unit_price_6	double	Price of unit 6	INFERRED	Pairs with unit6. See unit_price_1.
credit_account	string	GL account code credited for this item	INFERRED	Which postings use it (sales, returns) is undocumented. No account lookup endpoint (Q42). Sensitive (chart of accounts).
debit_account	string	GL account code debited for this item	INFERRED	Which postings use it is undocumented. No account lookup endpoint (Q42). Sensitive (chart of accounts).
curr1	string	Currency code of price 1	INFERRED	Whether it belongs to sale_parice1 (price class 1) or unit_price_1 (unit 1) is unknown (ISCO-29b). Presumably Currency.code, e.g. 'NIS' (ISCO-28b).
curr2	string	Currency code of price 2	INFERRED	See curr1: class 2 or unit 2 is unknown.
curr3	string	Currency code of price 3	INFERRED	See curr1: class 3 or unit 3 is unknown.
curr4	string	Currency code of price 4	INFERRED	See curr1: class 4 or unit 4 is unknown.
curr5	string	Currency code of price 5	INFERRED	See curr1: class 5 or unit 5 is unknown.
curr6	string	Currency code of price 6	INFERRED	See curr1: class 6 or unit 6 is unknown.
orgnization_code	string	Organization code; meaning not documented	UNDOCUMENTED	[sic 'organization'], name kept. Also on Store; branch or company is unknown (ISCO-27c). No lookup endpoint (Q42).

Data dictionary: every record and field

Stock quantity of an item in one store كمية الصنف في المستودع `ItemStore`

`/Item/GetItemsStores, /Item/getItemDetails · p. 11`

One row per item per store (warehouse or van) with the quantity, shelf and unit (p. 11). Returned in bulk by /Item/GetItemsStores ('Get all items quantities', p. 10; filters: lastupdatedate and itemcode headers, no store filter) and per item as ItemDetails.itmStores.

Field	Type	Meaning	Source	Note
`item_code`	`string`	Item code	INFERRED	→ Item.code. Filtered by GetItemsStores header itemcode, all lower-case (p. 10; ISCO-13).
`store`	`int`	Store (warehouse or van) id	INFERRED	→ Store.id from /Files/GetStores (class printed as 'City', p. 6). GetItemsStores cannot filter by store (ISCO-36d).
`qnty`	`double`	Quantity of the item in this store	INFERRED	Endpoint text 'Get all items quantities' (p. 10). Presumably in main_unit (ISCO-36a); zero or negative rows and reservations by open orders are undocumented (ISCO-36b,c).
`shelf`	`string`	Shelf / bin location in the store	INFERRED	From the field name.
`main_unit`	`string`	Unit the quantity is counted in (main unit)	INFERRED	Presumably equals Item.unit1 (ISCO-36a).

One item with its stock per store تفاصيل صنف معين وكمياته `ItemDetails` `/Item/getItemDetails · p. 11`

A wrapper holding one item's full card plus its quantity in every store, looked up with the itemCode header (p. 11); it serves the p. 2 capability 'pull quantities of a specific item' (سحب كمات [sic] صنف معين). The manual says 'It return list of this structure' but the shape is a single object, so callers must accept either (Q32).

Field	Type	Meaning	Source	Note
`itm`	`Item`	The item's full card (read model)	INFERRED	Printed 'Itemitm;' with the space missing. Same 51-field shape as GetItems rows.
`itmStores`	`List<ItemStore>`	The item's quantity in each store	INFERRED	Same rows GetItemsStores returns for this item (assumed).

Data dictionary: every record and field

Item group (category) مجموعة أصناف `ItemGroup` /Item/GetItemsGroups · p. 11

A node in the item category tree with its parent code and level (p. 11). Items point to it through item_group_code and item_group_father_code. /Item/GetItemsGroups has no inputs and is not in the p. 2 capability list.

Field	Type	Meaning	Source	Note
code	string	Group code; items refer to it	MANUAL	Comment: 'Item has itemGroupCode field that refer to this field' (p. 11). The real Item field is item_group_code (Q28, ISCO-35c); the write-model group presumably takes this code.
name	string	Group name	INFERRED	From the field name; mirrored in Item.item_group_name.
parentCode	string	Code of the parent group	INFERRED	→ ItemGroup.code. The Item model calls it item_group_father_code (ISCO-35c). Value for top-level groups (empty or null) is undocumented.
level	int	Depth of the group in the tree	INFERRED	Whether the top level is 0 or 1 is undocumented.

Item image reference كتالوج الأصناف `ItemCatalog` /Item/GetItemsCatalog · p. 11

Pairs an item code with the location of its picture ('Get all items image', p. 11). Only a path string comes back; its format, and how to fetch the image itself, are undocumented (ISCO-34). /Item/GetItemsCatalog has no item filter.

Field	Type	Meaning	Source	Note
item_code	string	Item code	INFERRED	→ Item.code. No item filter: all items come back in one call.
image_path	string	Location of the item's image	INFERRED	From 'Get all items image' (p. 11). URL, local path or network (UNC) path is unknown, and no image-bytes endpoint is documented (Q26, ISCO-34). May expose internal server paths.

Data dictionary: every record and field

Serial-numbered unit of an item الرقم التسلسلي للصنف `ItemSerial` `/Item/GetItemsSerials · p. 12`

One serial-numbered unit of an item with its status, store and warranty (guarantee) start and end dates (p. 12). Returned by /Item/GetItemsSerials ('Get all items with serials only', p. 11), filterable by customer code and serial headers; not in the p. 2 capability list.

Field	Type	Meaning	Source	Note
`name`	`string`	Item name (assumed)	INFERRED	Could also be a label for the serial unit; undocumented.
`itemCode`	`string`	Item code	INFERRED	→ Item.code. camelCase, unlike item_code elsewhere (Q45).
`serial`	`string`	Serial number of this unit	INFERRED	Filtered by header serial ('serials with this serial code only', p. 11). GetItems also has a serial header (ISCO-35a).
`status`	`int`	Status of the serial; values not documented	UNDOCUMENTED	E.g. in stock vs sold is a guess (ISCO-49). The customer header ('serials with this customer code only', p. 11) filters by customer, yet no customer field is returned.
`store`	`int`	Store holding this serial	INFERRED	→ Store.id from /Files/GetStores.
`start_grantee_date`	`string`	Warranty (guarantee) start date	INFERRED	[sic 'guarantee'], name kept. Date as a string, presumably yyyy-MM-dd (p. 4).
`end_grantee_date`	`string`	Warranty (guarantee) end date	INFERRED	[sic 'guarantee'], name kept. Presumably yyyy-MM-dd (p. 4).

Last-price report row (item pricing history) سطر تقرير آخر الأسعار `LastPriceRepColumn`

`/Report/GetLastPriceReport · p. 12`

One row of the 'last prices' report (تقرير اخر الأسعار, p. 3) for one item: date, document code, price, currency, quantity, bonus, unit and customer (p. 12). Inputs are headers ItemCode ('mandatory') and barcode; the manual's 'ItemCode is optional' next to barcode contradicts itself (Q21, ISCO-37a).

Field	Type	Meaning	Source	Note
`date`	`string`	Date of the transaction	INFERRED	String; presumably yyyy-MM-dd (p. 4).
`code`	`string`	Document (voucher) code, presumably	INFERRED	Voucher code is unconfirmed (ISCO-37d). The row has no item-code field because the report is for the item in the ItemCode header.
`price`	`double`	Unit price in that transaction	INFERRED	Whether net of discount or VAT is undocumented. Which voucher types count (sales only?) is unknown (ISCO-37d).
`currency`	`string`	Currency of the price	INFERRED	Presumably Currency.code (ISCO-28b).
`quantity`	`double`	Quantity in that transaction	INFERRED	Presumably in the unit given by unit.
`bonus`	`double`	Free bonus quantity	INFERRED	By analogy with VoucherItems.bonus 'كمية البونص' (bonus quantity), p. 14.
`unit`	`string`	Unit name the price applies to	INFERRED	A string here, while VoucherItems.unit is an int (p. 14).
`customer`	`string`	Customer of the transaction (code or name)	INFERRED	Printed 'stringcustomer' with the space missing. Code vs name unknown; no customer filter, so whose prices are returned is unclear (Q21, ISCO-37b,c).

Data dictionary: every record and field

New item (write) صنف جديد `Item (write)` → /Item/Post · p. 20

The JSON object sent in form field obj to /Item/Post to 'Insert new Item' (pp. 19-20); the manual prints its class name as plain 'Item'. Every field has an Arabic comment; it is a smaller, differently named set than the read model (one unit, one price, one currency), and the caller supplies the item code.

Field	Type	Meaning	Source	Note
code	string	Item number (supplied by the caller)	MANUAL-AR	Comment 'رقم الصنف' (item number). Whether an existing code is rejected is unknown (ISCO-43d).
unit1	string	Item's main unit	MANUAL-AR	Comment 'وحدة الصنف الرئيسية' (item's main unit). No unit2..unit6 can be set.
name	string	Item name	MANUAL-AR	Comment 'اسم الصنف' (item name). No English-name field (read nameEn).
type	int	Item type	MANUAL-AR	Comment 'نوع الصنف' (item type). Allowed values undocumented (ISCO-48b, ISCO-49).
price1	double	Price of the main unit	MANUAL-AR	Comment 'سعر الوحدة الرئيسية' (main-unit price). Whether it becomes sale_parice1 or unit_price_1 is unclear (Q29, ISCO-29b). Only one price can be set.
currency	string	Currency of the price	MANUAL-AR	Comment 'العملة' (currency). Expected values unconfirmed (ISCO-48b); presumably Currency.code. Read model has curr1..curr6.
group	string	Group the item belongs to	MANUAL-AR	Comment 'تابع لصنف' (literally 'belongs to item [category]'). Presumably ItemGroup.code, read back as item_group_code (ISCO-48b).
barcode	string	Barcode	MANUAL-AR	Comment 'الباركود' (barcode).
original_code	string	Original number	MANUAL-AR	Comment 'الرقم الاصلي' (original number). The read model spells it orginal_code (Q29).
factory_code	string	Manufacturer's (factory) number	MANUAL-AR	Comment 'رقم المصنع' (factory number).
trademark	int	Trademark / brand id	MANUAL-AR	Comment 'العلامة التجارية' (trademark). Numeric id with no lookup endpoint; not returned by any read (Q42, ISCO-48c).
producing_company	int	Producing (manufacturing) company id	MANUAL-AR	Comment 'الشركة المنتجة' (producing company). No lookup endpoint; write-only (Q42, ISCO-48c).
style	int	Style id	MANUAL-AR	Comment 'الطراز' (style). No lookup endpoint; write-only (Q42, ISCO-48c).

New item (write) (continued)

Field	Type	Meaning	Source	Note
model	int	Model id	MANUAL-AR	Comment 'الموديل' (model). GetItems has a model filter header, but no read returns the field (Q29, ISCO-35a).
note	string	Item note	MANUAL-AR	Comment 'ملاحظة النصف' [sic: ملاحظة الصنف, item note] (Q30). Not returned by any read.

Data dictionary: every record and field

Voucher (complete document) الحركة الكاملة (السند) `Voucher`

`/Voucher/GetVoucher, /Voucher/GetVouchers, /Voucher/GetDelivery, → /Voucher/Post2 · p. 13`

One accounting or stock document of any of the 13 voucher types (invoice, return, receipt, payment, order, journal, delivery...), made of a header plus item, cheque and account lines. Printed as class Voucher on p. 13 (Post2 input) and p. 17 (GetVoucher output); GetDelivery (p. 16) returns a variant without accounts, GetVouchers says only 'array of Voucher' without naming the class (ISCO-40a), and GetExtraVouchers uses a different short class of the same name (see ExtraVoucher).

Field	Type	Meaning	Source	Note
`header`	`VoucherHeader`	Voucher information: type, number, date, customer, totals	MANUAL-AR	«معلومات الحركة» = voucher information. See VoucherHeader.
`items`	`List<VoucherItems>`	Item lines of the voucher	MANUAL-AR	«تفاصيل اصناف الحركة» = details of the voucher's items. Used by item-based types: invoices, returns, orders, deliveries, stock-out (inferred). See VoucherItems.
`cheqs`	`List<VoucherCheqs>`	Cheques attached to the voucher	MANUAL-AR	«الشيكات» = the cheques. Typically receipts and payments (inferred; ISCO-47b). See VoucherCheqs.
`accounts`	`List<VoucherAccounts>`	Account (ledger) lines; mandatory only in journal vouchers	MANUAL-AR	«الحسابات، اجبارية في سند القيد فقط» = accounts, mandatory in journal vouchers (Journal = 28) only. Not present in the GetDelivery variant (p. 16). Whether allowed in other types: ISCO-47a. See VoucherAccounts.

Data dictionary: every record and field

Voucher header رأس الحركة `VoucherHeader`

`/Voucher/GetVoucher, /Voucher/GetVouchers, /Voucher/GetDelivery, → /Voucher/Post2 · p. 13`

The top part of every voucher: its type, number, date and time, currency, customer, salesman, totals and status flags (pp. 13–14). The same header is used for all voucher types; some fields only apply to orders or invoices.

Field	Type	Meaning	Source	Note
`type`	`int`	Voucher type (see VoucherTypes)	MANUAL-AR	«نوع الحركة» = voucher type. Must be one of VoucherTypes (p. 15, stated in English).
`code`	`string`	Voucher number; Shamel assigns one if not sent	MANUAL-AR	«رقم الحركة، اذا لم يتم ارسال رقم الحركة يتم اعطائها رقم من الشامل» = voucher number; if no number is sent, Shamel gives it one. If filled it must start with the type's prefix letter (p. 15, VoucherTypes.code_prefix). Format after the letter and uniqueness unknown (ISCO-45). Post2 returns the created number in Result.refrenceId.
`date`	`string`	Voucher date (yyyy-MM-dd)	MANUAL-AR	«تاريخ الحركة» (yyyy-MM-dd) = voucher date.
`time`	`string`	Voucher time (hhmmss)	MANUAL-AR	«وقت الحركة» (hhmmss) = voucher time. 24-hour clock assumed, not stated (ISCO-24c).
`currency`	`string`	Currency code, e.g. NIS or $	MANUAL-AR	«كود العملة» (eg. "NIS","$") = currency code. Links to Currency.code (inferred); codes are not ISO 4217.
`rate`	`double`	Exchange rate of the voucher currency	MANUAL-AR	«سعر صرف العملة» = currency exchange rate.
`customer_code`	`string`	Customer number	MANUAL-AR	«رقم الزبون» = customer number. Links to Customer.code (inferred).
`city`	`int`	City of the voucher	INFERRED	No comment in the manual. Probably City.id from /Files/GetCities (inferred from name and int type).
`manual_code`	`string`	Paper reference number, if any	MANUAL-AR	«يتم استخدامه لتدوين رقم المرجعي الورقي ان وجد» = used to record the paper reference number, if one exists. Probably what the GetStatus header 'manual' expects (inferred; ISCO-42a). Whether duplicates are rejected is unknown (ISCO-43b).
`is_posted`	`bool`	Enter the voucher as posted or not	MANUAL-AR	«هل يتم ادخال حالة الحركة مرحلة ام لا» = whether the voucher is entered with status posted or not. Effect of each value: ISCO-46e.
`is_printed`	`bool`	Enter the voucher as printed or not	MANUAL-AR	«هل يتم ادخال حالة الحركة مطبوعة ام لا» = whether the voucher is entered with status printed or not.

Data dictionary: every record and field

Voucher header (continued)

Field	Type	Meaning	Source	Note
discount	double	Total invoice discount (an amount)	MANUAL-AR	«قيمة الخصم الكلي للفاتورة» = value of the invoice's total discount. An amount, whereas VoucherItems.discount is a percentage. Invoice-type vouchers (per the comment).
balance	double	Invoice value with the invoice discount	MANUAL-AR	«قيمة الفاتورة مع خصم الفاتورة» = invoice value with the invoice discount. Unclear whether net of the discount, and whether it must be sent or is computed (ISCO-46a, ISCO-46d).
update_date	string	Date the voucher was last updated	INFERRED	No comment; meaning read from the name. Format not stated; whether to fill on insert unknown.
note	string	Note	MANUAL-AR	«ملاحظة» = note.
note2	string	Second note	MANUAL-AR	«ملاحظة 2» = note 2.
vat	double	Tax (VAT) amount	MANUAL-AR	«قيمة الضريبة» = tax value. Presumably the voucher total, while VoucherItems.vat is per unit (inferred; ISCO-46c).
delivery_id	string	Number of the consignment (on-sale) delivery	MANUAL-AR	«رقم الارسالية برسم البيع» = number of the delivery sent 'for sale' (consignment). Probably used for 'invoice from delivery' and type 5 returns (inferred; ISCO-47c, ISCO-47g).
salesman	string	Salesman (sales rep) number	MANUAL-AR	«رقم المندوب» = salesman number. Probably the salesman code (AuthResult.salesmanCode; /Files/GetSalesmen is undocumented) (inferred).
cheq_balance	double	Total value of the cheques	MANUAL-AR	«مجموع قيمة الشيكات» = sum of the cheques' value, i.e. total of the cheqs list (inferred). Receipts and payments (inferred; ISCO-47b).
date_to_deliver	string	Delivery date for the order	MANUAL-AR	«تاريخ التسليم للطلبية» = delivery date for the order. Orders (type 0).
time_to_deliver	string	Delivery time for the order	MANUAL-AR	«وقت التسليم للطلبية» = delivery time for the order. Orders (type 0). Format not stated.
payment_type	int	Payment type (values undocumented)	MANUAL-AR	«نوع الدفعة» = payment type. Values not documented and no lookup endpoint (ISCO-49).

Data dictionary: every record and field

Voucher header (continued)

Field	Type	Meaning	Source	Note
order_store	int	Store (warehouse) of the order	MANUAL-AR	the order's store. Links to Store.id = «مستودع الطلبية» from /Files/GetStores (inferred). Orders (type 0).
customer_balance	double	Customer balance at voucher time, when requested	INFERRED	No comment. Filled when header getCustomerBalance=true on GetVoucher/GetVouchers; otherwise '0 is returned' (p. 17), so 0 is ambiguous (ISCO-31). GetVouchers excludes the voucher itself; GetVoucher does not say. Not meaningful on insert (inferred).
system_id	int	Number between 1000 and 2000; purpose not stated	MANUAL	'value must be between 1000 and 2000'. Purpose undocumented; probably identifies the integrating system. Whether 1000/2000 are allowed is unknown (ISCO-44).

Data dictionary: every record and field

Voucher item lines أصناف الحركة `VoucherItems`

`/Voucher/GetVoucher, /Voucher/GetVouchers, /Voucher/GetDelivery, → /Voucher/Post2 · p. 14`

One line per item in a voucher: item, unit, store, quantity, price, bonus, discount, tax and up to six cost centers (p. 14). Some fields are filled by Shamel only (voucher_code, name) and some are not used (measure, color, serials).

Field	Type	Meaning	Source	Note
`voucher_code`	`string`	Voucher number this line belongs to	MANUAL-AR	«رقم الحركة، لا يجب تعبئته في الادخال» = voucher number; do not fill on insert.
`pos`	`int`	Line order in the voucher, starting from 0	MANUAL-AR	«ترتيب الصنف في الحركة، يبدا من صفر» = the item's order in the voucher, starts from zero.
`code`	`string`	Item number	MANUAL-AR	«رقم الصنف» = item number. Links to Item.code (inferred).
`name`	`string`	Item name	MANUAL-AR	«اسم الصنف، لا يجب تعبئته في الادخال» = item name; do not fill on insert (output only).
`unit`	`int`	Item unit	MANUAL-AR	«وحدة الصنف» = item unit. Probably an index 1–6 into Item.unit1..unit6 (inferred; ISCO-49).
`store`	`int`	Store (warehouse) of the line	MANUAL-AR	«المستودع» = the store. Links to Store.id from /Files/GetStores (inferred).
`measure`	`int`	Item size; not used	MANUAL-AR	«قياس الصنف، غير مستخدم» = item size/measure; not used.
`color`	`int`	Item colour; not used	MANUAL-AR	«لون الصنف، غير مستخدم» = item colour; not used.
`quantity`	`double`	Item quantity	MANUAL-AR	«كمية الصنف» = item quantity.
`price`	`double`	Price	MANUAL-AR	«السعر» = the price. Presumably per unit (inferred).
`bonus`	`double`	Bonus (free) quantity	MANUAL-AR	«كمية البونص» = bonus quantity.
`discount`	`double`	Line discount percentage, 0–100	MANUAL-AR	«نسبة الخصم» = discount percentage. A 0-100 percentage, whereas VoucherHeader.discount is an amount.
`vat`	`double`	Tax amount for a quantity of 1	MANUAL-AR	«قيمة الضريبة لكمية 1» = tax value for a quantity of 1 (per unit).
`sold_qnty`	`double`	Sold quantity	MANUAL-AR	«الكمية للباعة» = the quantity sold. Probably the part of a consignment delivery already sold (inferred).

Data dictionary: every record and field

Voucher item lines (continued)

Field	Type	Meaning	Source	Note
delivery_pos	double	Line number of the item in the consignment delivery	MANUAL-AR	«رقم الصنف في الارسالية برسم البيع» = the item's number in the 'for sale' delivery. Pairs with VoucherHeader.delivery_id (inferred). A position typed as double.
note	string	Item note	MANUAL-AR	«ملاحظة اصنف» [ملاحظة الصنف sic, for] = item note.
serials	string[]	Serial numbers; not used	MANUAL-AR	«غير مستخدم» = not used.
costcenter1	string	Cost-center code, slot 1	INFERRED	No comment. Probably CostCenterDTO.code; six cost-center slots per line. No endpoint lists cost centers (ISCO-39b).
costcenter2	string	Cost-center code, slot 2	INFERRED	No comment. See costcenter1.
costcenter3	string	Cost-center code, slot 3	INFERRED	No comment. See costcenter1.
costcenter4	string	Cost-center code, slot 4	INFERRED	No comment. See costcenter1.
costcenter5	string	Cost-center code, slot 5	INFERRED	No comment. See costcenter1.
costcenter6	string	Cost-center code, slot 6	INFERRED	No comment. See costcenter1.

Voucher cheques شيكات الحركة `VoucherCheqs`

`/Voucher/GetVoucher, /Voucher/GetVouchers, /Voucher/GetDelivery, → /Voucher/Post2 · p. 14`

One line per cheque attached to a voucher: cheque number, account, bank, branch, due date, currency and amount (p. 14).

Field	Type	Meaning	Source	Note
cheq_num	string	Cheque number	MANUAL-AR	«رقم الشيك» = cheque number.
account_num	string	Account number	MANUAL-AR	«رقم الحساب» = account number. Probably the bank account the cheque is drawn on (inferred).
bank_code	string	Bank number	MANUAL-AR	«رقم البنك» = bank number. Links to Bank.code from /Files/GetBanks (inferred).
branch_code	string	Bank branch number	MANUAL-AR	«رقم فرع البنك» = bank branch number. Links to BankBranch.code from /Files/GetBanksBranchs (inferred).
due_date	string	Cheque due date	MANUAL-AR	«تاريخ الاستحقاق» = due date. Format yyyy-MM-dd (p. 4).
rate	double	Exchange rate of the cheque currency	INFERRED	No comment; read from the name.
currency	string	Cheque currency code	INFERRED	No comment. Probably Currency.code (inferred).
amount	double	Cheque amount	MANUAL-AR	«قيمة الشيك» = cheque value.

Data dictionary: every record and field

Voucher account lines (journal lines) حسابات الحركة (أسطر القيد) `VoucherAccounts`

`/Voucher/GetVoucher, /Voucher/GetVouchers, → /Voucher/Post2 · p. 14`

One ledger line per account in a voucher: account, amount, debit/credit side, six cost centers and a note (pp. 14–15). Mandatory only in journal vouchers (p. 13). The manual gives no comments for this class.

Field	Type	Meaning	Source	Note
`position`	`int`	Line order in the voucher	INFERRED	No comment. Named pos in VoucherItems; whether it starts at 0 here is not stated.
`account_code`	`string`	General-ledger account code	INFERRED	No comment. No endpoint lists GL accounts, so codes cannot be validated through the API.
`amount`	`double`	Line amount	INFERRED	No comment; read from the name.
`debit_credit`	`int`	Debit or credit side (values undocumented)	INFERRED	No comment; meaning from the name. Numeric values not documented (ISCO-49).
`costcenter1`	`string`	Cost-center code, slot 1	INFERRED	No comment. Probably CostCenterDTO.code (ISCO-39b).
`costcenter2`	`string`	Cost-center code, slot 2	INFERRED	No comment. See costcenter1.
`costcenter3`	`string`	Cost-center code, slot 3	INFERRED	No comment. See costcenter1.
`costcenter4`	`string`	Cost-center code, slot 4	INFERRED	No comment. See costcenter1.
`costcenter5`	`string`	Cost-center code, slot 5	INFERRED	No comment. See costcenter1.
`costcenter6`	`string`	Cost-center code, slot 6	INFERRED	No comment. See costcenter1.
`note`	`string`	Line note	INFERRED	No comment; read from the name.

Voucher insert result نتيجة إدخال الحركة `Result (voucher)` `/Voucher/Post2 · p. 15`

The answer to /Voucher/Post2, printed as class Result (p. 15): a success flag, a list of error messages and the number of the voucher that was created. A different class also named Result is returned by /CostCenter/Post.

Field	Type	Meaning	Source	Note
`Success`	`bool`	True if the voucher was inserted	INFERRED	No comment; read from the name and 'If the process succeeded' (p. 15). Errors can also arrive as HTTP 500 (p. 4).
`ErrorMessage`	`string[]`	List of error messages	INFERRED	No comment. An array here, a single string in PostResponse and in Result (cost center).
`refrenceId`	`string`	Number (code) of the inserted voucher	MANUAL	[sic: reference] 'If the process succeeded, the refrenceId will contain the inserted code of the voucher'. Arabic «رقم الحركة التي تم ادخالها» = number of the voucher that was inserted. Use it with GetVoucher (ISCO-40d).

Data dictionary: every record and field

Voucher summary row ملخص الحركة `ExtraVoucher` /Voucher/GetExtraVouchers · p. 18

A short, flat voucher row returned by /Voucher/GetExtraVouchers, printed as 'public class Voucher' (p. 18); the name ExtraVoucher is assigned here to tell it apart from the full Voucher. It has amount, direction and the customer's balance but no lines. What 'extra' means is not explained (ISCO-40b).

Field	Type	Meaning	Source	Note
`type`	`int`	Voucher type (see VoucherTypes)	INFERRED	No comment; presumably the same values as VoucherHeader.type.
`code`	`string`	Voucher number	INFERRED	No comment; read from the name.
`date`	`string`	Voucher date	INFERRED	No comment. Format yyyy-MM-dd (p. 4).
`customer_code`	`string`	Customer number	INFERRED	No comment. Links to Customer.code (inferred).
`amount`	`double`	Voucher amount	INFERRED	No comment; read from the name. Gross or net not stated.
`currency`	`string`	Currency code	INFERRED	No comment. Probably Currency.code (inferred).
`rate`	`double`	Exchange rate	INFERRED	No comment; read from the name.
`cr_dr`	`int`	Credit or debit direction (values undocumented)	INFERRED	No comment; meaning from the name. Numeric values not documented (ISCO-49).
`customer_balance_at_voucher_date`	`double`	Customer balance at the voucher date, excluding this voucher	INFERRED	Matches the header getCustomerBalance text on p. 18: balance 'at the invoice time (excluding the invoice itself)'; 0 if the header is not set or false.

Voucher list filters فلاتر قائمة الحركات `VoucherFilters` → /Voucher/GetVouchers, → /Voucher/GetExtraVouchers · p. 17

An unnamed JSON object sent in the request header 'filters' of /Voucher/GetVouchers and /Voucher/GetExtraVouchers (p. 17, repeated pp. 17–18); the name VoucherFilters is assigned here. There is no filter by voucher type and no paging (ISCO-40c, ISCO-21).

Field	Type	Meaning	Source	Note
`customerCode`	`string`	Only vouchers of this customer	INFERRED	Marked 'optional' in the manual. Links to Customer.code (inferred).
`currencyCode`	`string`	Only vouchers in this currency	INFERRED	Marked 'optional' in the manual. Links to Currency.code (inferred).
`fromDate`	`string`	Start of the date range (yyyy-MM-dd)	INFERRED	Not marked optional. Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.
`toDate`	`string`	End of the date range (yyyy-MM-dd)	INFERRED	Not marked optional. Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.

Data dictionary: every record and field

Customer statement filters فلاتر كشف الحساب المحاسبي `AccountsheetFilters (customer)`

→ /Report/GetAccountsheet · p. 12

The filters of the customer account statement report (كشف حساب محاسبي, p. 3), printed as class AccountsheetFilters on p. 12 and sent as JSON in the form field 'obj' of /Report/GetAccountsheet. A different class with the same name is used by the cost-center statement. No field has a comment.

Field	Type	Meaning	Source	Note
customerCode	string	Customer (account) whose statement is requested	INFERRED	Read from the name. Links to Customer.code (inferred). Whether other account codes are accepted: ISCO-38a.
currencyCode	string	Statement currency	INFERRED	Read from the name. Links to Currency.code (inferred).
fromDate	string	Period start date (yyyy-MM-dd)	INFERRED	Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.
toDate	string	Period end date (yyyy-MM-dd)	INFERRED	Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.
showOtherCurrencies	bool	Include movements in other currencies	INFERRED	Read from the name.
showStocks	bool	Include item/stock details	INFERRED	Read from the name; exact effect not stated.
showCheqs	bool	Include cheques	INFERRED	Read from the name.
showNotPayedCheqs	bool	Include unpaid cheques	INFERRED	Read from the name. [sic: 'Payed' for 'Paid'].
showReturnedCheq	bool	Include returned (bounced) cheques	INFERRED	Read from the name. Singular 'Cheq', unlike the sibling fields.

Data dictionary: every record and field

Cost-center statement filters فلاتر كشف حساب مركز التكلفة `AccountsheetFilters (cost center)`

→ /CostCenter/GetCostCenterAccountsheet · p. 19

The filters of the cost-center account statement report (تقرير كشف حساب لمركز تكلفة, p. 3), printed as class AccountsheetFilters on p. 19 and sent as JSON in the form field 'filters' (not 'obj') of /CostCenter/GetCostCenterAccountsheet. No field has a comment.

Field	Type	Meaning	Source	Note
`fromCostcenter`	`string`	First cost center of the range	INFERRED	Read from the name. Links to CostCenterDTO.code (inferred).
`toCostcenter`	`string`	Last cost center of the range	INFERRED	Read from the name. Links to CostCenterDTO.code (inferred).
`account`	`string`	Account code to report on	INFERRED	Read from the name. What it accepts (GL account, customer): ISCO-39a.
`type`	`int`	Undocumented filter type	UNDOCUMENTED	No comment and no values given (ISCO-39a, ISCO-49). An int here, a string in CostCenterDTO.type.
`currencyCode`	`string`	Statement currency	INFERRED	Read from the name. Links to Currency.code (inferred).
`fromDate`	`string`	Period start date (yyyy-MM-dd)	INFERRED	Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.
`toDate`	`string`	Period end date (yyyy-MM-dd)	INFERRED	Format yyyy-MM-dd (p. 4). Inclusive or not: ISCO-24b.
`showOtherCurrencies`	`bool`	Include movements in other currencies	INFERRED	Read from the name.

Statement line سطر كشف الحساب `AccountsheetColumns`

/Report/GetAccountsheet, /CostCenter/GetCostCenterAccountsheet · p. 13

One row of an account statement, returned as a list by both the customer statement and the cost-center statement (printed identically on pp. 13 and 19). All amounts are strings with an unstated number format (ISCO-38b). No field has a comment.

Field	Type	Meaning	Source	Note
`date`	`string`	Movement date	INFERRED	Read from the name.
`code`	`string`	Voucher number of the movement	INFERRED	Probably VoucherHeader.code (inferred).
`type`	`string`	Voucher type of the movement	INFERRED	A string here, an int in vouchers; whether name, code or number is unknown (ISCO-38d, ISCO-49).
`credit`	`string`	Credit amount	INFERRED	Amount sent as text; number format unknown (ISCO-38b).
`debit`	`string`	Debit amount	INFERRED	Amount sent as text; number format unknown (ISCO-38b).
`start_date`	`string`	Undocumented date	UNDOCUMENTED	No comment; possibly a due or value date (guess). ISCO-38c.
`balance`	`string`	Running balance after this movement	INFERRED	Amount sent as text. Whether the opening balance is a separate first row is unknown (ISCO-38e).
`note`	`string`	Movement note	INFERRED	Read from the name.

Data dictionary: every record and field

Cost center (new) مركز تكلفة جديد `CostCenterDTO` → /CostCenter/Post · p. 18

A new cost center to insert, sent as JSON in the form field 'obj' of /CostCenter/Post (p. 18). The manual wrongly titles the endpoint 'Insert new customer' (copy-paste error; p. 2 lists 'مركز تكلفة جديد', new cost center). No field has a comment, and no endpoint lists existing cost centers (ISCO-39b).

Field	Type	Meaning	Source	Note
code	string	Cost-center code	INFERRED	Read from the name. Client-supplied; the only reference to the new cost center since no id is returned (ISCO-39d). Used by costcenter1..6 in voucher lines (inferred).
name	string	Cost-center name (Arabic)	INFERRED	Read from the name; Arabic assumed because name_en exists.
name_en	string	Cost-center name in English	INFERRED	Read from the name.
father	string	Parent cost-center code	INFERRED	Read from the name (father = parent in a tree). ISCO-39c.
type	string	Undocumented cost-center type	UNDOCUMENTED	No values given (ISCO-39c, ISCO-49). A string here, an int in the cost-center statement filter.
date	string	Undocumented date	UNDOCUMENTED	Purpose not stated (ISCO-39c). Presumably yyyy-MM-dd per p. 4.

Cost-center insert result نتيجة إدخال مركز التكلفة `Result (cost center)` /CostCenter/Post · p. 19

The answer to /CostCenter/Post, printed as class Result (p. 19): only a success flag and one error message. Unlike Result (voucher) and PostResponse, it returns no identifier.

Field	Type	Meaning	Source	Note
Success	bool	True if the cost center was inserted	INFERRED	No comment; read from the name. Errors can also arrive as HTTP 500 (p. 4).
ErrorMessage	string	Error text	INFERRED	No comment. A single string here, an array in Result (voucher).

Data dictionary: every record and field

Enumeration: VoucherTypes p. 15

Value	Name	Meaning	Arabic	Prefix
9	SalesInvoice	Sales invoice	فاتورة مبيعات	I
4	ReturnSalesInvoice	Sales return (goods returned by a customer)	مردودات مبيعات	S
5	ReturnedPendingSalesDelivery	Return from a consignment (on-sale) delivery	مردود من ارسالية برسم البيع	F
24	Receipt	Receipt voucher (money received)	قبض	R
27	Payment	Payment voucher (money paid out)	صرف	P
0	Order	Sales order	طلبية	O
22	Credit	Credit note	اشعار دائن	C
23	Debit	Debit note	اشعار مدين	D
28	Journal	Journal entry	قيد	J
16	InternalDelivery	Internal delivery (transfer between stores, inferred)	ارسالية داخلية	N
2	PurchaseDelivery	Purchase delivery (goods received)	ارسالية مشتريات	
12	Stockout	Goods issue (stock-out)	اخراج بضاعة	B
10	SalesDelivery	Sales delivery (goods dispatched for sale)	ارسالية مبيعات	V

Enumeration: OrderStatus p. 16

Value	Name	Meaning	Arabic	Prefix
0	not ready	Not ready		
1	ready	Ready		
2	closed	Closed		
3	sent partially	Sent partially		
4	sent	Sent		

C APPENDIX

Open questions that would widen the scope

The manual leaves gaps that change what we can safely build. 55 questions (Arabic and English) are ready to send to ISCO; these ten matter most.

Id	Topic
`ISCO-01`	Session token lifetime and concurrent sessions per user
`ISCO-05`	HTTPS / TLS support
`ISCO-18`	Error format, stable error codes, expired-token signal
`ISCO-20`	Rate and concurrency limits
`ISCO-21`	Pagination, truncation and large responses
`ISCO-22`	Meaning of lastupdatedate
`ISCO-26`	Response of /Files/GetSalesmen
`ISCO-43`	Duplicate prevention and outcome of failed writes
`ISCO-50`	Shamel version, API versioning, change notices
`ISCO-53`	Licensing: internal layer now, commercial use later

Where they are

`docs/design/08-isco-questions.md` (full, bilingual, with cover letter) and `08-isco-questions-short.md` (56 short English questions, copy-paste ready). Every question was checked against this manual.

How answers change this document

Answers on update endpoints, paging, change feeds, enum values and undocumented fields would move several “partial” and “blocked” tool ideas in section 11 to “ready”.